



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

석사학위논문

롤셋 및 증분 재분석(ΔT) 기반

TARA 자동화 시스템

- ISO/SAE 21434 준수 TARA 산출물의
감사추적성과 재현성 보장 -

고려대학교 SW·AI 융합대학원

소프트웨어보안학과

윤민희

2026년 2월

이 희 조 교수 지도

석사학위논문

롤셋 및 증분 재분석(ΔT) 기반

TARA 자동화 시스템

- ISO/SAE 21434 준수 TARA 산출물의

감사추적성과 재현성 보장 -

이 논문을 공학 석사학위 논문으로 제출함

2025 년 10 월

고 려 대 학 교 SW·AI 융 합 대 학 원

소프트웨어보안학과

윤 민 희 (인)



윤민희의 공학 석사학위논문 심사를 완료함

2025 년 12 월

위원장 이 희 조 (인)

위 원 우 승 훈 (인)

위 원 전 유 석 (인)



룰셋-증분 재분석(ΔT) 통합 기반 TARA 자동화 시스템

윤 민 희

소프트웨어보안학과

지도교수: 이 희 조

초록

Software-Defined Vehicle(SDV) 환경에서는 빈번한 OTA(Over-The-Air) 업데이트로 인해 위험 분석 및 위험 평가(TARA)의 재분석 빈도가 급증하고 있다. 그러나 기존 TARA 도구들은 사소한 변경에도 전체 시스템을 재분석하는 방식을 사용하여, 개발 일정 지연과 사이버보안 위험 분석의 처리 성능을 제약하는 주요 요인이 된다.

본 연구는 이러한 문제를 해결하기 위해 변경된 요소만 선택적으로 재분석하는 증분 재분석 엔진(ΔT 엔진)을 제안한다. 제안 시스템은 STPA-STRIDE 통합 방법론 기반의 룰셋 추론, 방향성 비순환 그래프(DAG) 기반 의존성 모델링, SHA-256 해시 기반 변경 감지, 선택적 재계산 전파의 핵심 메커니즘으로 구성된다. 또한 감사 번들(Audit Bundle)을 통해 ISO/SAE 21434 가 요구하는 결정론적 재현성과 추적성을 보장한다.

실험은 차량 전장 제어기(ICU) 도메인 데이터셋을 대상으로 네 가지 연구 질문을



검증하였다. RQ1(실행 가능성)은 산출물 및 SHA-256 해시 비교를 통해 증분 재분석 모드가 전체 재분석 모드와 동일한 결과를 생성함을 확인하였다. RQ2(효율성)는 9 개 실험 구성에서 평균 5.57 배, 최대 12.15 배의 Speedup 을 달성하였다($p < 0.001$). RQ3(정확도 및 재현성)은 변경 영향 노드를 98% 이상 정확히 탐지하였고, 동일 조건 반복 실행 시 SHA-256 해시 100% 일치로 결과 재현성을 확보하였다. RQ4(적용 조건)는 Dirty Ratio (전체 노드 중 재계산이 필요한 노드의 비율) 기반 Zone 별 운용 가이드라인을 도출하였다.

본 연구는 TARA 증분 재분석의 실용성을 실증적으로 검증하였으며, SDV 환경에서 TARA 프로세스의 지속 가능한 적용을 위한 기술적 기반을 제공한다.

중심어: TARA, 증분 재분석, ISO/SAE 21434, Software-Defined Vehicle, 의존성 그래프, 위협 분석



Integration-Based TARA Automation System

Combining Rule Sets and Incremental Reanalysis (ΔT)

by Minhee Yun

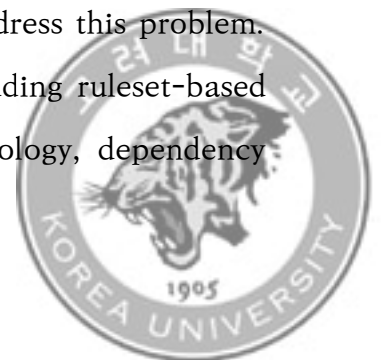
Department of Software Security

under the supervision of Professor Heejo Lee

ABSTRACT

In Software-Defined Vehicle (SDV) environments, frequent Over-The-Air (OTA) updates have led to a significant increase in the frequency of Threat Analysis and Risk Assessment (TARA) re-execution. However, conventional TARA tools perform full system re-analysis even for minor changes, causing development delays and limiting the processing efficiency of cybersecurity threat analysis.

This study proposes an incremental re-analysis engine (ΔT engine) that selectively re-analyzes only the changed elements to address this problem. The proposed system comprises core mechanisms including ruleset-based inference built on an integrated STPA-STRIDE methodology, dependency



modeling based on Directed Acyclic Graph (DAG), SHA-256 hash-based change detection, and selective recalculation propagation. Furthermore, the Audit Bundle ensures deterministic reproducibility and traceability as required by ISO/SAE 21434.

Experiments were conducted on an Integrated Control Unit (ICU) domain dataset to validate four research questions. RQ1 (Feasibility) confirmed that the incremental re-analysis mode generates identical results to the full re-analysis mode through artifact and SHA-256 hash comparison. RQ2 (Efficiency) achieved an average speedup of $5.57\times$ and a maximum of $12.15\times$ across nine experimental configurations ($p < 0.001$). RQ3 (Accuracy and Reproducibility) demonstrated over 98% accuracy in detecting change-affected nodes and achieved 100% SHA-256 hash consistency across repeated executions under identical conditions. RQ4 (Application Conditions) derived zone-based operational guidelines based on Dirty Ratio, defined as the proportion of nodes requiring recalculation among total nodes.

This study empirically validates the practicality of incremental TARA re-analysis and provides a technical foundation for sustainable application of the TARA process in SDV environments.

Keywords: TARA, Incremental Re-analysis, ISO/SAE 21434, Software-Defined Vehicle, Dependency Graph, Threat Analysis



감사의 글

대학원 생활의 첫걸음부터 마침표를 찍는 이 순간까지, 늘 동행하시며 선한 길로 인도해주신 하나님께 감사드립니다.

부족한 저를 깊은 이해와 세심한 지도로 이끌어 주신 이희조 교수님께 깊은 존경과 감사를 표합니다. 연구의 방향을 잃고 헤맬 때마다 따뜻한 격려와 날카로운 조언으로 방향을 제시해 주신 덕분에 연구를 마무리할 수 있었습니다. 교수님의 가르침은 연구자로서 가져야 할 자세와 태도를 배울 수 있는 소중한 시간이었으며, 교수님과 함께 연구할 수 있었던 모든 순간이 영광이었습니다.

언제나 변함없는 사랑과 믿음으로 저를 응원해 준 부모님에게 감사드립니다. 지치고 힘든 순간마다 묵묵히 곁을 지켜주고, 제가 선택한 길을 존중하며 응원해주신 부모님이 계셨기에 여기까지 올 수 있었습니다. 이 작은 결실이 그 동안의 사랑에 조금이나마 보답이 되길 바랍니다.

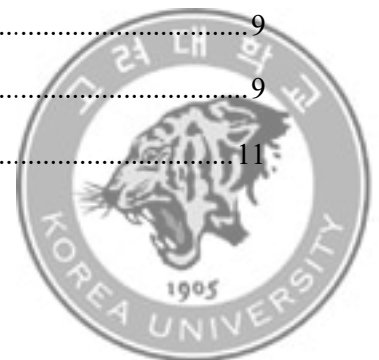
마지막으로, 대학원 선후배, 동기 분 들에게도 고마운 마음을 전하고 싶습니다. 힘들 때 서로를 격려하고 기쁠 때 함께 나누며, 서로의 연구에 관심을 가지고 조언을 아끼지 않았던 모든 순간들이 이 여정을 완주할 수 있게 한 원동력이었습니다. 각자의 연구 주제는 달랐지만 같은 목표를 향해 나아가며 만들어간 연대감은 더 넓은 시각으로 학문을 바라볼 수 있었고, 여러분과 함께한 시간은 제 인생에서 가장 소중한 추억으로 남을 것입니다. 앞으로도 서로의 든든한 동료로 함께하기를 기대합니다.

이 논문은 저 혼자 힘의 아님, 많은 분들의 도움과 격려가 모여 이루어진 결실입니다. 이 배움을 바탕으로 더욱 성장하는 연구자가 되어 받은 은혜에 보답하겠습니다.



목차

초록.....	i
ABSTRACT	iii
감사의 글.....	v
목차.....	vi
표 목차.....	ix
그림 목차.....	xi
제 1 장. 서론	1
1.1 연구의 필요성.....	1
1.2 연구 범위와 목적.....	3
제 2 장. 관련 연구와 한계	5
2.1 STPA-STRIDE 통합 접근 개요.....	5
2.1.1 제어 구조 기반 접근: STPA.....	5
2.1.2 위협 유형 기반 접근: STRIDE.....	5
2.1.3 공격 경로 시각화 접근: 공격 트리.....	6
2.1.4 방법론 통합의 필요성.....	6
2.1.5 STPA-STRIDE 통합 모델 연구.....	7
2.2 TARA 자동화 도구 비교.....	8
2.3 한계 정리: 증분 재분석 부재, 재현성·추적성 미흡.....	8
2.3.1 증분 재분석 부재	8
2.3.2 재현성·추적성 미흡.....	9
2.4 기존 연구 분석 요약 및 시사점	9
제 3 장. 룰셋-증분 재분석(ΔT) 통합 기반 TARA 자동화 시스템	11



3.1 시스템 아키텍처.....	11
3.2 시스템 핵심 메커니즘.....	13
3.2.1 룰셋 기반 위협 식별 엔진.....	13
3.2.2 증분 재분석(ΔT).....	14
3.2.3 결정론적 재현성 및 감사 가능성 보장.....	17
3.3 데이터 모델 및 추적성 체계.....	18
3.3.1 ID 생성 정책 및 핵심 스키마.....	19
3.3.2 데이터 모델의 논리적 연결 구조.....	20
3.3.3 양방향 추적성 및 영향 분석.....	20
3.4 감사 가능성과 표준 정합성 검증 메커니즘.....	21
제 4 장. 실험 설계와 방법.....	23
4.1 실험 대상 및 데이터셋.....	23
4.2 실험 시나리오 정의.....	24
4.3 실험 설계 및 비교 방법.....	25
4.3.1 실험 모드 비교: Full vs ΔT	25
4.3.2 연구 전략 및 실행 절차.....	26
4.4 평가 지표 공식.....	27
4.4.1 성능 지표.....	27
4.4.2 정확도 및 재현성 지표.....	27
4.4.3 커버리지 및 표준 정합성 지표.....	28
4.5 실험 설계의 타당성.....	29
제 5 장. 실험 결과 및 논의.....	31
5.1 RQ1 검증: ISO/SAE 21434 산출물 생성.....	31
5.2 RQ2 검증: 증분 재분석 성능 향상.....	32
5.3 RQ3 검증: 증분 재분석의 정확도 및 재현성.....	34
5.4 RQ4 검증: 성능 예측 모델 및 적용 조건.....	35
5.5 실험 결과의 통계적 검증.....	37

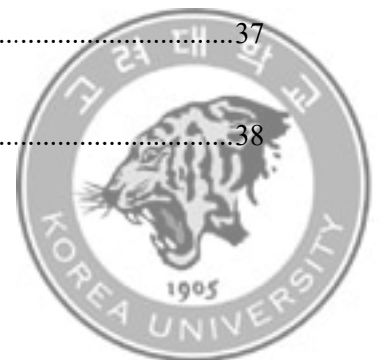


제 6 장. 결론	39
6.1 결론.....	39
6.1.1 연구의 기여.....	39
6.1.2 실무적 함의.....	40
6.1.3 연구의 한계.....	40
6.2 향후 과제	41
참고문헌	43



표 목차

〈표 1〉 산업 도메인별 특성 비교와 TARA 수행 난이도.....	3
〈표 2〉 자동화 관점에서 본 TARA 관련 방법론 비교.....	7
〈표 3〉 TARA 접근법 및 도구의 자동화 및 변경관리/감사성 수준 비교.....	10
〈표 4〉 감사 가능성의 구성 요소.....	18
〈표 5〉 ID 생성 규칙 및 예시.....	19
〈표 6〉 감사 번들 구성 요소.....	21
〈표 7〉 실험 시나리오와 연구 질문 매핑.....	25
〈표 8〉 산출물 유형별 생성 현황.....	31
〈표 9〉 표준 적합성 메트릭 결과.....	31
〈표 10〉 변경 영향 분석의 정확도 검증 결과.....	35
〈표 11〉 실험 결과 통계 검정 요약.....	37
〈표 12〉 정확도 메트릭 요약.....	37
〈표 13〉 전파 깊이별 성능-정확도 상충 분석.....	38



〈표 14〉 Zone 기반 운용 전략.....	40
---------------------------	----



그림 목차

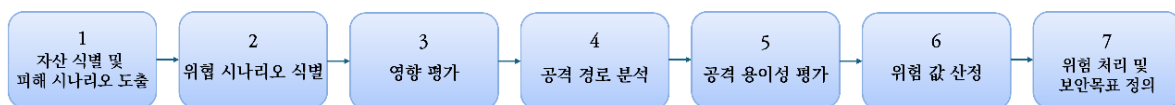
(그림 1) TARA 7 단계 프로세스 (Clause 15.2).....	1
(그림 2) TARA 자동화 시스템 아키텍처.....	11
(그림 3) ΔT 엔진의 변경 전파 메커니즘.....	16
(그림 4) 감사 가능성 구현을 위한 메커니즘 간 관계	21
(그림 5) Full 모드 vs. ΔT 모드 실행 시간 비교 (9 개 구성).....	32
(그림 6) 변경률에 따른 Speedup 변화 (스케일별).....	33
(그림 7) 시스템 규모 및 변경 비율에 따른 속도 향상 분석	34
(그림 8) Dirty Ratio 증가에 따른 속도 향상 변화 및 효율성 구간 분석.....	35



제 1 장. 서론

1.1 연구의 필요성

커넥티비티 강화와 복잡한 E/E 아키텍처로 자동차의 새로운 사이버보안 위협이 대두되었다. 자동차 사이버보안의 심각성은 2015년 Jeep Cherokee 원격 해킹 사건으로 널리 인식되었으며, 이 사건은 140만 대 리콜과 함께 자동차 사이버보안 규제 강화의 계기 중 하나가 되었다[1]. 이에 대응해 UNECE WP.29 R155와 ISO/SAE 21434가 제정되었다. ISO/SAE 21434는 차량 사이버보안을 위한 국제 표준으로, 자동차의 전 생애 주기에 걸친 사이버보안 프로세스와 필수 요구사항을 정의한다. 이 표준의 핵심 요소는 자동차 사이버보안 목표를 도출하는 출발점인 Clause 15에 정의된 위협 분석 및 위협 평가(Threat Analysis and Risk Assessment; 이하 TARA)이다. TARA는 7단계로 구성된 핵심 프로세스로서, 차량의 사이버보안 위협을 체계적으로 식별, 분석, 평가 및 처리하는 접근 방식이다[10].



(그림 1) TARA 7 단계 프로세스 (Clause 15.2)

- (1) 자산 식별(Asset Identification): 보호해야 할 시스템 자산을 도출하고 그 보안 속성(기밀성, 무결성, 가용성)을 설정한다.
- (2) 위협 시나리오 도출(Threat Scenario Identification): 각 자산에 따라 발생 가능한 잠재적인 위협 시나리오를 식별한다.
- (3) 영향 평가(Impact Rating): Safety, Financial, Operational, Privacy 4 개 축에서 피해 영향도를 평가한다.
- (4) 공격 경로 식별(Attack Path Analysis): 위협이 실제로 실현 가능한 공격 경로를 구체적으로 분석한다.
- (5) 공격 용이성 평가(Attack Feasibility Rating): 특정 위협 시나리오에 대해 공격이 성공할 가능성을 전문 지식, 시간, 장비, 기회 등을 고려해서 평가한다.



- (6) 위험 결정(Risk Determination): 공격에 성공할 경우 시스템, 사용자, 조직에 미치는 영향도와 공격 용이성을 조합하여 최종 위험 수준을 산정한다.
- (7) 위험 처리(Risk Treatment): 각 위험에 대해 위험 수용, 회피, 경감, 이전 등 대응 방안 결정 및 사이버보안 요구사항 도출한다.

ISO/SAE 21434 표준은 TARA 산출물에 대해 추적성, 일관성, 변경 영향 평가를 요구한다. 특히 위험 시나리오에는 자산, 침해 속성 및 원인을 문서화하고, 공격 경로는 해당 시나리오와 연결되어 양방향 추적이 보장되어야 한다[11].

기존 TARA 방법론으로는 이러한 표준 요구사항을 충족하는 데 여러 제약이 따른다. SDV(Software-Defined Vehicle)로의 전환과 함께 차량 개발 주기에 애자일 개발 방법론이 도입되면서 개발 기간은 단축되고 있다. 그러나 빈번한 소프트웨어 업데이트는 TARA 재분석의 부담을 크게 증가시켜, 개발 일정 지연과 품질 저하로 이어지고 있다. 산업 현장에서 TARA를 수작업으로 수행하는 경우, 분석 결과가 분석가의 경험과 직관에 크게 의존하게 된다. 이로 인해 분석 결과의 일관성이 확보되지 않으며, 차량의 다양한 운행 상태에서 발생 가능한 공격 시나리오를 누락하거나 중복 분석하는 문제가 빈번하게 발생한다. 이와 같은 문제는 금융, IT, 의료 등 다양한 산업 분야에서 공통적으로 관찰되지만 자동차 산업에서는 타 산업에서 검증된 접근법을 직접 적용하는데 본질적인 제약이 존재한다.

첫째, 자동차의 보안 침해는 탑승자의 생명을 위협하는 안전사고로 전이될 수 있다[1]. 이에 따라 자동차 OEM은 ISO 26262와 ISO/SAE 21434를 동시에 충족해야 한다. 이러한 이중 규제 체계는 단일 규제를 따르는 타 분야와 구별되는 고유한 복잡성을 야기한다.

둘째, 자동차 산업의 공급망은 다층적이고 분산된 구조적인 특성을 갖는다. 현대의 차량은 100 개 이상의 ECU가 수십 개의 공급업체로부터 조달되어 일관된 보안 수준의 유지 및 관리가 어렵다[5]. 이로 인해 단일 공급업체의 취약점이 전체 시스템의 보안 취약점으로 확대될 수 있는 위험을 내포한다.

셋째, 자동차는 평균 11.8 년의 운용 수명 동안 통제되지 않는 외부 시설에 상시 노출된다[5]. 이러한 물리적 접근 가능성은 공격 벡터의 지속적 확대를 의미하고, 통제된 환경 기반의 보안 전략을 그대로 적용하는데 근본적인 한계로 작용한다.

<표 1>은 이러한 특성을 산업별로 비교 분석한 결과를 제시한다. 결론적으로 자동차는 이 세 가지 특성이 동시에 중첩되는 복합 시스템으로써 타 산업 분야의 보안 방법론을 직접 차용하는 것이 어렵다.



〈표 1〉 산업 도메인별 특성 비교와 TARA 수행 난이도

특성	자동차	금융/IT	의료	항공	TARA 수행의 어려움
동적 상태 전환 (주행/정차/충전/정비)	O	X	X	△	상태 조합에 따른 시나리오의 기하급수적 증가
이질적 공급망 통합	O	△	△	O	변경 영향 범위 예측 어려움
안전-보안 이중 규제	O	X	△	O	복잡한 추적성 관리부담
물리-사이버 융합 인터페이스	O	X	X	△	위협 분석기준의 모호성

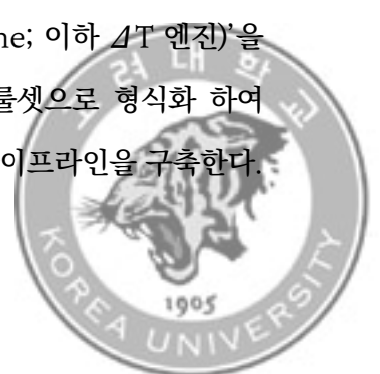
참고문헌 [2] ~ [9], 범례: O=완전해당, △=부분해당, X=비해당

1.2 연구 범위와 목적

본 연구는 ISO/SAE 21434 표준에 기반한 TARA를 효율적으로 수행하는 방법을 다루고 있으며, 특히 OTA 업데이트로 인해 시스템 구성이 빈번히 변경되는 환경에서 전체 재분석 없이 변경 영향 범위를 선택적으로 분석하는 데 초점을 맞춘다. ISO/SAE 21434 표준이 요구하는 위협 시나리오와 위협 평가를 다루고 있으며, ISO 26262 단독 분석이나 칩투 테스트 기반 취약점 평가는 연구 범위에서 제외된다.

연구 범위는 자동차의 핵심 구성요소인 통합제어장치 (Integrated Control Unit; 이하 ICU) 도메인으로 한정한다. ICU는 ISO 26262 ASIL-D, ISO/SAE 21434 CAL 3-4 등급이 요구되는 고신뢰성 컴포넌트로서, 이를 대상으로 한 검증 결과는 차량 시스템 전체로 확장 가능한 대표성을 지닌다. 또한, STPA는 계층적 제어 구조를 기반으로 작동하기 때문에 ICU의 명확한 제어 흐름은 본 연구에서 제안하는 STPA-STRIDE 통합 방법론을 검증하기에 적합하다. 특히 현재 조직에서 개발 중인 ICU의 실제 데이터를 활용함으로써, 합성 데이터 기반 검증의 한계를 극복하고 실증적 타당성을 확보하였다.

앞서 제시한 TARA 방법론의 한계인 TARA 수행 시 발생하는 분석가별 편차를 최소화하고, 변경 시 전체 재분석 부담을 감소시키면서 기존 수준의 분석 정확도를 유지하기 위해 본 연구에서는 ‘규칙 기반 자동화 파이프라인’과 ‘증분 재분석 엔진 (Delta-T incremental reanalysis engine; 이하 ΔT 엔진)’을 통합한 시스템을 제안한다. STRIDE와 STPA의 구조화된 분석 절차를 YAML 룰셋으로 형식화 하여 ISO/SAE 21434 표준을 준수하는 TARA 산출물을 자동으로 생성하는 규칙 기반 파이프라인을 구축한다.



또한 레코드 단위인 행 단위 변경 감지와 의존성 그래프 기반 전파 메커니즘을 활용한 LT 엔진을 개발하여 선택적으로 재분석을 수행한다.

본 연구는 기존 TARA 방법론 및 자동화 도구들이 지닌 정적 분석의 한계와 재분석의 비효율성을 식별한다. 이를 보완하기 위해 제안하는 방법론은 실제 ICU 개발 데이터를 활용한 실증적 검증을 거침으로써, 이론적 차별성 확보는 물론 급변하는 차량용 소프트웨어 업데이트 환경에서 사이버 보안 분석의 지속성과 신뢰성을 확보하는데 기여할 것이다.



제 2 장. 관련 연구와 한계

2.1 STPA-STRIDE 통합 접근 개요

2.1.1 제어 구조 기반 접근: STPA

STPA(System-Theoretic Process Analysis)는 시스템의 상태, 제어 프로세스를 분석해서 시스템 안전 위험을 체계적으로 식별하고 관리하는데 중점을 둔다[12].

시스템을 단순한 부품 고장이 아니라 시스템의 제어 구조와 요소 간 상호 작용에 초점을 맞추어, 계층적 제어 구조로 모델링 한다. STPA에서 UCA(Unsafe Control Action)는 시스템의 제어 행동(Control Action) 중에서 특정 상황이나 조건에서 위험을 초래할 수 있는 잠재적인 위험 요소로 간주하고, 시스템의 Loss와 Hazard에 명시적으로 연결되어 인과관계를 구조적으로 분석한다. 위험 상황에서 필수적인 제어 행위가 발생하지 않는 경우, 정상 상황에서 위험을 유발하는 제어 행위가 발생하는 경우, 제어 행위가 잘못된 타이밍 및 순서로 실행되는 경우, 제어 행위가 의도한 것과 다른 시간만큼 지속되는 경우로 발생한다. 그러나 STPA는 본질적으로 '안전(Safety)' 중심으로 설계되어, 악의적인 의도를 가진 '보안(Security)' 위협을 직접적으로 도출하는 데는 한계가 있다. 특히 스푸핑, 변조와 같은 의도적 공격 시나리오는 STPA 만으로는 충분히 식별되지 않는다[13].

2.1.2 위협 유형 기반 접근: STRIDE

STRIDE는 시스템 설계 단계에서 보안 취약점을 선제적으로 발견하기 위해 Microsoft가 개발한 프레임워크[14]로, 데이터 흐름 중심의 6가지 위협 범주 분석을 통해 체계적인 위협 모델링을 지원한다.

- (1) Spoofing (스푸핑): 공격자가 신원을 위조하여 인증을 우회하는 위협이다.
- (2) Tampering (변조): 승인되지 않은 행위로 데이터의 무결성을 훼손하는 위협이다.
- (3) Repudiation (부인): 범죄 행위에 대해 그 사실을 부인하여 책임 회피가 가능한 경우이다.
- (4) Information Disclosure (정보 노출): 비인가자에게 민감한 정보가 노출되는 경우이다.
- (5) Denial of Service (서비스 거부): 시스템 리소스를 고갈시키거나 의도적으로 정상적인 서비스 이용을 방해하는 공격이다.



- (6) Elevation of Privilege (권한 상승): 권한이 없는 사용자가 더 높은 권한을 획득해서 시스템을 제어하는 경우를 말한다.

STRIDE는 소프트웨어 보안 분야에서 널리 활용되며, ISO/SAE 21434 표준에서도 차량 사이버보안 위협 식별 방법론으로 권장된다. 하지만, STRIDE는 위협을 범주별로 분류하는 정적 분석 방식으로 인해 위협의 동적인 상태 변화나 구성 요소 간의 상호 의존성을 충분히 반영하지 못하는 한계가 있다[15]. 선행 연구에서 STRIDE 위협 모델링 프레임 워크를 단독으로 적용한 결과 총 228건의 위협 시나리오가 도출되었다. 그러나 이 중 약 67%만이 시스템 Hazard와 연결되고(정밀도 67%), 나머지 33%는 시스템의 동적 컨텍스트를 고려하지 못해 과도한 위협으로 분류되었다[16].

2.1.3 공격 경로 시각화 접근: 공격 트리

공격 트리는 공격자가 목표를 달성하기 위해 사용할 수 있는 공격 경로를 논리적인 트리 구조로 시각화한다[17]. 이 트리의 루트 노드는 공격자의 최종 목표이고, 하위 노드들은 그 목표를 이루기 위한 세부 공격 단계를 의미한다. 이때 AND, OR 논리 게이트를 통해 복합 공격 조건을 표현함으로써 다양한 공격 시나리오를 분석할 수 있다 [18]. 그러나 공격 트리는 정적인 공격 경로 표현에 적합하며, 공격 기술의 변화에 따라 트리를 지속적으로 업데이트 해야 하므로 동적인 변화에 신속하게 대응하기 어렵다. 또한, 시스템이 복잡해질수록 트리 구조가 매우 방대해지기 때문에 관리가 어렵고, 특정 공격이 발생하는 시간적 순서나 시스템의 상태 변이에 따른 공격 가능성 변화를 효과적으로 표현하는 데는 한계가 있다[19]. 이를 보완하기 위해 Attack-Defense Trees, Timed Attack Trees 등의 확장 모델이 제안되었으나, 여전히 STPA의 UCA 'Context'와 같은 세밀한 시스템 상태 조건을 포괄적으로 다루기는 어렵다[20].

2.1.4 방법론 통합의 필요성

앞서 살펴본 위협 분석 기법들을 Jeep Cherokee 사례(2015)[1]를 통해 분석한 결과, 자동차와 같은 사이버-물리 시스템에서 한계가 더욱 명확히 드러남을 알 수 있다. 이 사례는 특정 보안 방법론만으로는 해킹을 차단할 수 없음을 명확히 증명하고, 계층적이고 통합적인 보안 접근이 필수적임을 나타낸다. 구체적으로, STPA를 단독으로 적용한 경우 '고속 주행 중 긴급 제동 미 실행'이라는 UCA(Unsafe



Control Action)를 도출할 수 있지만, 우발적 고장과 의도적 공격을 구분할 수 없다. 동일한 시스템에 STRIDE 만을 적용한 경우 Spoofing, Tampering, DoS 등 보안 위협을 체계적으로 식별할 수 있으나, 차량의 동적 운행 상태를 반영하지 못해 과잉 식별 문제를 야기한다. 또한, 공격 트리만을 사용하면 ‘도심 저속 주행 → 고속도로 진입 → 자율주행 활성화’와 같은 동적 상태 전이를 표현할 때 트리 크기가 기하급수적으로 증가함에 따라 상태 공간의 조합적 폭발이 발생하며, 이는 시스템 확장성을 저해하는 주요 요인이 된다.

따라서 단일 방법론만으로는 복잡한 차량 시스템의 보안 위협을 완전히 분석할 수 없으며, 각 방법론의 강점을 결합한 통합 접근법이 필요하다. <표 2>는 이러한 관점에서 세가지 방법론의 특성을 비교한 것이고 본 연구의 해결책은 3 장에서 제시한다.

<표 2> 자동화 관점에서 본 TARA 관련 방법론 비교

방법론	접근 방식	자동화 적합 요인	자동화 제약 요인
STPA	제어 구조와 안전 제약 중심의 하향식 분석	• 체계적이고 구조화된 분석 절차 • 시스템 전체 관점의 일관성 유지	• 도메인 맥락 이해 필수 • 전문가 판단 의존성 높음 • 완전 자동화 달성 어려움
STRIDE	DFD 요소와 6 개 위협 범주 간 매핑 기반의 상향식 체크리스트 방식	• 명확한 분류 체계 • 룰셋 기반 자동화 용이 • 패턴 매칭 적용 가능	• 고유 위험 평가 체계 부재 (CVSS 등 외부 메트릭 통합 필요) • 맥락 기반 판단 제한적
공격 트리	공격 목표와 하위 경로의 계층적 모델링	• 그래프 구조 기반 알고리즘 적용 용이 • 경로 탐색 자동화 가능	• 조합 폭발 문제 • 대규모 시스템 확장성 제약

2.1.5 STPA-STRIDE 통합 모델 연구

최근 연구들은 Safety 와 Security 를 동시에 고려하는 접근법인 STPA 의 시스템 이론적 분석 능력과 STRIDE 의 체계적 보안 위협 식별 능력을 통합한 연구 결과를 제시하고 있다. 이러한 통합 접근법은 개발 초기 단계에서 안전 요구사항과 함께 기밀성, 무결성, 가용성 침해로 인한 보안 손실을 파악해서 분석 작업의 중복을 줄이고 일관성을 높이는 효과를 보여주었다 [21].

STPA-Sec(System-Theoretic Process Analysis for Security)는 STPA 를 보안 분석으로 확장한 대표적인 방법론이지만, 많은 연구에서 STPA 의 한계가 지적되었다. 의도적인 공격에 대한 인과 시나리오에 대한 가이드 부족, 프로세스 제어와 직접 연결되지 않는 데이터 교환을 분석 범위에서 제외하며,



기밀성 등 정보보안 중심 속성을 충분히 다루지 못한다는 한계가 지적되었다[22]. 이러한 한계를 극복하기 위해 STPA-DFSec(Data-flow-based adaption of STPA-Sec) 등 다양한 확장 모델들이 제안되었으며, 차량 시스템의 특수성을 고려한 통합 연구도 진행되고 있다[23]. 그러나 이러한 통합 모델들도 시스템 변경에 따른 증분 재분석 문제는 해결하지 못했다.

2.2 TARA 자동화 도구 비교

현대 차량 시스템의 복잡성 증가로 인해 TARA 프로세스 자동화는 필수 요구사항이다. 최대 150 개의 ECU, 다양한 인터페이스, 외부 연결성, 다중 네트워크를 포함한 차량 아키텍처는 수동 분석과 추적성 관리를 현실적으로 어렵게 한다[24].

최근 연구들은 TARA 자동화를 위해 형식적 검증 기반 접근법과 LLM 기반 접근법을 제시하였다. SysML 기반 환경인 AVATAR는 안전과 보안 관련 요소를 동일한 모델에서 반영하고 형식적으로 검증할 수 있는 방법을 제공하나[25], 형식적 모델 작성의 복잡성과 전문성을 요구하는 한계가 있다. 최근 Defense Weaver[26]는 대규모 언어 모델(LLM)을 활용하여 개별 기능 단위로 TARA 분석을 자동화하고, 새로운 위협과 다양한 표준에 대한 적응성을 제공한다. 그러나 LLM의 확률적 토큰 생성 메커니즘으로 인해 동일 입력에 대한 결과 재현성을 보장하지 못하는 근본적인 한계를 지닌다.

ISO/SAE 21434 준수를 위한 상용 TARA 도구들은 TOE 및 E/E 아키텍처 모델링, 위협 분석, 위협 평가 기능을 제공하며, 최근에는 요구사항부터 테스트까지의 추적성이 강화되고 있다[27]. 그러나 이들은 변경 시 영향 범위의 선택적 평가 없이 전체 재분석을 수행하여, 빈번한 변경이 발생하는 SDV 환경에서 과도한 비용과 시간을 요구한다.

2.3 한계 정리: 증분 재분석 부재, 재현성·추적성 미흡

2.3.1 증분 재분석 부재

TARA 자동화 도구들은 ISO/SAE 21434가 요구하는 변경 이력 추적과 버전 관리 기능을 제공하지만, 대부분 직접 연결된 위협만 식별할 뿐 의존성 전파를 통한 간접 영향을 파악하지 못해 전체 재분석이 불가피하다.



주요 전기차 제조사들이 월 평균 0.8 ~ 1.2 회 정도 소프트웨어 업데이트를 배포할 때, OTA 업데이트를 통해 차량 전체 시스템의 5~15%의 정도만 업데이트된다. 이러한 환경에서 표준 준수를 위해 연간 7~14 회의 TARA 재수행이 요구된다. 따라서 매번 전체 재분석을 수행하거나 변경 영향을 과소평가하여 위험 누락 위험이 있는 기존 도구들은 실무 적용에 한계를 보인다.

선행 연구[28]에서는 위험 간 상호 의존성을 나타내기 위해 전제 조건과 후 조건 개념을 사용하며, 일부 위험의 후조건이 다른 위험의 전제조건이 될 수 있음을 강조한다. 이러한 개념을 기반으로 효율적인 재분석을 위해 위험 간의 의존성을 식별하고 특정 요소의 변경이 의존성 체인을 따라 어떻게 전파되는지 추적하는 메커니즘이 필수적이다[29].

다른 산업 분야에서는 증분 재분석이 일반적으로 활용되고 있으나, 자동차 TARA 연구는 위험 탐지 정확도 향상에 편중되어 설계 변경 시 재분석에 대한 효율적인 방안을 제시하지 못하고 있다.

2.3.2 재현성·추적성 미흡

상용 TARA 도구들은 최종 결과를 문서 형태로 출력하는 기능에 집중되어 있다. 그 결과 데이터 간 연결 관계가 압축된 형태로 제공되기 때문에 추적성 확보와 결과 도출의 인과 관계 파악이 제한적이다. 특히, 메타데이터를 체계적으로 수집하고 관리하는 기능이 부재하여 동일 입력에 대한 결과 일관성을 보장하지 못한다.

한편, LLM 기반 접근법은 확률적 토큰 생성을 통한 위험 탐색에는 유용하나, 표준의 재현성 요구사항과 근본적으로 상충한다. 이러한 한계들로 인해 ISO/SAE 21434 가 요구하는 재현성과 추적성을 충족하지 못하며, 궁극적으로 TARA 결과의 신뢰성을 저해한다. 따라서, 분석 프로세스 전반에 걸쳐 데이터 추적성과 재현성을 보장하는 체계적인 접근법이 요구된다.

2.4 기존 연구 분석 요약 및 시사점

기존 TARA 접근법들은 위험 식별 방법을 개선하고 자동화했다는 점에서 의미 있는 성과를 달성하였다. 그러나 SDV 환경이 요구하는 변경 관리의 효율성, 결과 재현성, 데이터 무결성 측면에서는 공통적인 한계를 보인다. <표 3>은 각 접근법의 자동화 수준과 변경관리 및 감사 가능성을 비교 분석한 결과이다.



〈표 3〉 TARA 접근법 및 도구의 자동화 및 변경관리/감사성 수준 비교

접근법/도구	자동화	변경관리/ 감사성	핵심 특징	주요 한계
수동 TARA	1	1	전문가 주도, 상황별 맞춤	자동화 및 추적성 부재
Ansys medini analyze for Cybersecurity	5	3	ISO 21434 대응, 모델 기반	ΔT / 의존성 전파 미지원
ThreatGet	5	3	EA 연동, 변경 영향 표시	직접 연결 중심, ΔT 부재
Panasonic VERZEUSE™ series	5	4	전 주기 보안, 룰 기반	TARA / ΔT 명시 약함
DefenseWeaver (LLM)	9	2	높은 자동화	재현성 부재
본 연구 (제안 시스템)	9	9	룰셋+ ΔT +감사 번들, 데이터 추적	-

※ ΔT : 증분 재분석(Incremental Re-analysis)

※ 자동화: 위협 식별-평가-위험도 산출의 자동화 정도 및 도구 지원 여부

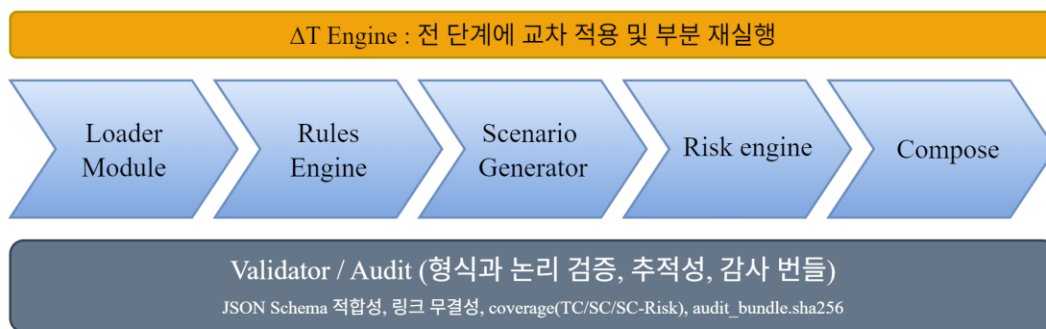
※ 변경관리/감사성: 증분 재분석, 감사 추적, 재현성 지원 여부



제 3 장. 룰셋-증분 재분석(ΔT) 통합 기반 TARA 자동화 시스템

3.1 시스템 아키텍처

본 연구에서 제안하는 TARA 자동화 파이프라인은 효율성, 일관성, 증분 재분석 시간 절감, 재현성의 실험적 검증을 위해 설계되었다. 시스템은 총 5단계 (Loader, Rules, Scenario, Risk, Compose)로 구성되며, 각 단계는 선행 단계의 분석 결과를 기반으로 위협 모델을 점진적으로 구체화한다.



(그림 2) TARA 자동화 시스템 아키텍처

(그림 2)는 본 연구에서 제안하는 TARA 자동화 시스템의 전체 아키텍처를 나타낸다.

본 시스템은 핵심 파이프라인과 부가 메커니즘으로 구성되며, 핵심 파이프라인은 UCA 입력부터 위협 평가 산출물 생성까지의 순차적 데이터 흐름을 담당하는 5단계로 이루어진다. 부가 메커니즘은 파이프라인 전체에 적용되는 요소와 외부에서 품질 보증을 수행하는 요소로 구분된다.

- (1) Loader 모듈: 사용자가 CSV, JSON 형식으로 제공한 파일들을 읽어서 사전에 정의된 데이터 스키마 규약에 맞게 각 행과 필드를 내장 파이썬 객체로 변환한다. 초기 단계에서 필수 필드 누락, 데이터 형식의 규칙 준수와 같은 기본적인 형식 검증을 수행한 후 다음 단계에서 이러한 문제로 에러가 발생하지 않도록 방지한다.



- (2) Rules Engine : Loader가 메모리에 적재한 UCA 및 시스템 인터페이스 객체들을 입력으로 받아 규칙 집합(Ruleset)을 실행한 후 위협 항목(TH)을 도출한다. 본 시스템은 평가 로직과 도메인 지식을 YAML 형식의 룰셋 파일로 외부화 하여 구현 코드로부터 분리한다. 이를 통해 보안 전문가가 개발자의 도움 없이 직접 평가 기준을 수정할 수 있고, 룰셋의 버전 관리를 통해 투명성, 감사 가능성, 재현성을 확보할 수 있다. 이러한 과정을 거쳐 생성된 최종 산출물은 STRIDE 범주로 분류된 위협 레코드 집합이다.
- (3) Scenario Generator: Rules Engine에서 생성한 위협 항목(TH)을 차량의 상태 전이 모델과 결합해, 실제로 발생할 수 있는 피해 시나리오를 생성한다. 이는 상태 전이 모델을 참조해서 각 위협이 어떤 상태에서 발현되고, 그 후 시스템이 연쇄적으로 어떻게 변경 가능한지 체계적으로 열거함으로써 중복되거나 불가능한 시나리오를 제거한다.
- (4) Risk Engine: 식별된 각 피해 시나리오(Damage Scenario; 이하 DS)에 대해 ISO/SAE 21434 요구사항을 충족하는 위험도(Risk)를 산정한다. 구체적으로, 영향도(Impact)는 Safety, Financial, Operational, Privacy의 4개 축에서 평가하고, 공격 용이성(Attack Feasibility)을 함께 산정한 후, 사전에 정의된 위험 매트릭스(risk_matrix.json)를 통해 이를 조합하여 최종 위험 등급을 결정한다. 이때, 산정 절차는 결정론적 특성을 가지며, 결과로 생성되는 위험 평가 레코드는 최종 위험 수준(등급 및 점수), 고유 식별자와 해당 피해 시나리오에 대한 참조를 속성으로 포함한다.
- (5) Compose 모듈: 앞선 단계들의 결과를 분석하여 최종 TARA 산출물로 통합한다. 통합 과정에서는 UCA, TH, DS, Risk 각 요소에 고유 ID를 할당하고, 사전에 설계한 데이터 스키마와 추적성 규칙에 따라 요소 간 연결 관계를 정의한다. 이러한 ID 기반 연결 구조를 통해 UCA부터 최종 위험까지 전체 흐름에 대한 양방향 추적이 가능해진다. Compose 모듈의 최종 산출물은 양방향 추적성이 확보된 통합 TARA 데이터셋(composed.json, risk.csv, trace_index.json)이다.

부가 메커니즘은 핵심 파이프라인의 순차적인 단계가 아니라, 파이프라인 전체에 걸쳐 교차 적용되거나 실행 전후에 동작하는 독립적 구성요소이다. 이들은 파이프라인의 데이터 흐름에 직접 참여하지 않으면서 성능 최적화와 품질 보증 기능을 담당한다.

- (1) Validator/Audit 모듈 (품질 보증 및 재현성) : 핵심 파이프라인 실행 완료 후에 동작하는 후처리 구성요소로서, 산출물의 품질 검증과 감사 증거 생성을 담당한다. (그림 2)에서 파이프라인 하



단에 위치한 'Validator/Audit' 표시가 이를 나타낸다. 이 모듈은 Export, Validator, Audit Bundle(이하 감사 번들) 생성의 세 가지 핵심 기능을 수행하며, Validator의 구조는 3.4절에서, 감사 번들 메커니즘은 3.2.3절에서 상세히 기술한다.

- (2) ΔT 메커니즘 : 핵심 파이프라인 전체에 걸쳐 교차적으로 적용되는 증분 처리 구성요소이다. 입력 데이터나 설정이 부분적으로 변경되었을 때 전체 재분석 없이 영향 받는 부분만 선택적으로 재계산함으로써 분석 성능을 향상시킨다. (그림 2)에서 파이프라인 상단에 위치한 ' ΔT Engine' 표시가 이를 나타낸다. 이 메커니즘은 방향성 비순환 그래프 (Directed Acyclic Graph, 이하 DAG)로 표현된 의존성 그래프 기반 영향 범위 분석, 해시 기반 변경 감지, 선택적 재계산 전파의 세 가지 핵심 기능을 통해 동작한다. 이를 통해 증분 재분석의 최종 결과는 완전 재분석과 동일함이 보장되며, 추적성 또한 유지된다. ΔT 엔진의 상세 알고리즘과 성능 평가는 3.2.2절에서 기술한다.

3.2 시스템 핵심 메커니즘

3.2.1 룰셋 기반 위협 식별 엔진

기존 상용 TARA 도구들은 핵심 평가 로직을 비공개 소프트웨어로 구현하지만 리스크 매트릭스 정의, 영향도 및 가능성 평가기준, 분석 규칙 등은 설정 파일을 통해 프로젝트 특성에 따라 구성 가능하다.

반면, 본 연구에서는 위협 식별 로직을 YAML 기반 룰셋으로 외부화 한다. 이러한 구조는 평가 엔진과 도메인 지식의 분리를 통해 코드 수정 없이 룰셋만으로 평가 기준을 갱신할 수 있게 한다. 또한, 룰셋의 버전 관리를 통해 평가 기준의 변경 이력을 기록하고, 과거 분석 결과의 재현이 가능하다. 이는 ISO/SAE 21434 에서 요구하는 추적성 및 문서화 요구사항을 충족하는 핵심 메커니즘이다.

룰셋의 각 규칙은 조건부 위협 식별을 위해 IF-THEN-UNLESS (when-then-unless) 구조를 따르며, Rules Engine 은 이를 기반으로 다음과 같은 처리 과정을 수행한다.

먼저 states.csv 와 interfaces.csv 로부터 시스템 경계와 보호 자산을 추출하여 분석 범위를 설정하고, 제어 계층 구조를 DAG 로 모델링한다. 이후 각 UCA 에 대해 다음 메커니즘이 순차적으로 적용된다.



첫째, 조건 매칭 단계에서 UCA 객체가 규칙의 when 절과 일치하는지 평가하며, STPA의 네 가지 UCA 조건(필요 제어 미 실행, 불필요 제어 실행, 타이밍 오류, 부적절한 지속)을 검사하고 시스템 Hazard와 연결한다. 둘째, 예외 검사 단계에서는 unless 절을 통해 특정 보안 조치가 적용되었거나 특수 운영 조건에서 위협이 무효화되는 상황을 처리한다. 셋째, 위협 생성 단계에서는 then 절 템플릿에 따라 구체적인 공격 시나리오를 포함한 위협 객체를 생성한다. 넷째, 충돌 해결 메커니즘을 통해 여러 규칙이 동일 UCA에 매칭될 경우, priority 필드와 명시적 unless 절을 통해 적용 순서를 결정하고 동일한 우선순위 간 충돌 시 경고 로그를 생성한다. 마지막으로 UCA 유형별로 STRIDE 범주를 자동 매핑하여 위협을 체계적으로 분류한다. 각 UCA에 대해 STRIDE의 여섯 가지 위협 범주를 적용하고, 단일 위협뿐만 아니라 시퀀스 다이어그램, 공격 트리, 상태 기반 조건 분석을 통해 다중 위협의 상호작용을 고려한 복합 시나리오를 도출한다 [16].

3.2.2 증분 재분석(ΔT)

기존 TARA 프로세스는 분석 단계 간의 순차적 의존성으로 인해 N 개의 제어 동작 상태와 M 개의 위협을 분석할 경우, 시간 복잡도는 $O(N \times M)$ 에 달한다. 실무에서 대부분의 변경은 국지성(locality of change)을 가지며, 전체 시스템 중 일부 요소만 수정되고 나머지는 유지된다. 그러나 기존 도구들은 이러한 변경의 국지성을 활용하지 못하고 전체 재분석을 수행하며 비효율성을 초래한다. SDV(Software-Defined Vehicle) 환경에서는 빈번한 OTA(Over-The-Air) 업데이트로 인해 TARA 재분석 빈도가 급증하여, 시스템 성능을 제약하는 주요 요인이 된다. 이에 본 연구는 DAG 기반의 의존성 전파 메커니즘을 구현하여 변경 영향 범위를 최소화하는 ΔT 엔진을 제안한다.

정의 3.1 TARA의 의존성 그래프

TARA 분석 과정의 의존성은 DAG $G = (V, E)$ 로 표현된다. 이는 위상 정렬(topological sort)을 통한 일관된 처리 순서 보장을 가능하게 한다. 실제 파이프라인에서는 $UCA \rightarrow TH \rightarrow DS \rightarrow Risk \rightarrow Trace$ 순서로 데이터가 흐르며, 각 단계의 산출물이 다음 단계의 입력이 된다.



- 정점 집합 V 는 다음과 같이 분할된다.

$$V = V_{UCA} \cup V_{TH} \cup V_{DS} \cup V_{Risk} \cup V_{trace}$$

V_{UCA} : Unsafe Control Action 노드 집합

V_{TH} : Threat 노드 집합

V_{DS} : Damage Scenario 노드 집합

V_{Risk} : Risk Assessment 노드 집합

V_{trace} : Traceability Index 노드 집합

- 간선 집합 $E \subseteq V \times V$ 는 노드 간 의존성을 정의하며, 간선 $(u, v) \in E$ 는 노드 u 에서 노드 v 로 영향 전파 경로를 나타낸다.

정의 3.2 변경 집합(Dirty-set)

시점 t 에서의 변경 집합 $D(t)$ 는 다음과 같이 정의된다.

$$D(t) = \{v \in V \mid H(v, t) \neq H(v, t-1)\}$$

여기서 $H(v, t)$ 는 시점 t 에서 노드 v 의 SHA-256 해시 값을 나타낸다. 실제 구현에서 각 노드의 해시 값은 해당 노드의 모든 속성을 정규화 된 형식(JSON)으로 직렬화 한 후 SHA-256을 적용하여 계산되고, 이를 통해 실제로 변경된 노드 만을 정확하게 식별 가능하다.

정의 3.3 영향 전파 함수

변경 전파 함수 P 는 그래프 $G = (V, E)$ 상에서 임의의 노드 부분집합 D 를 입력 받아, 의존성 전파를 통해 산출된 새로운 노드 부분집합 $P(D)$ 를 반환하는 전파 연산자이다.

$$P(D) = D \cup \{v \in V \mid \exists u \in D, (u, v) \in E^+\}$$

여기서 E^+ 는 간선 집합 E 의 추이적 폐포(transitive closure)를 의미한다. 직접 의존뿐 만 아니라 간접 의존 관계까지 모두 포함하여 영향 받는 노드를 계산한다.

ΔT 엔진은 $P(D(t))$ 를 효율적으로 근사하기 위해 다음 파라미터들을 활용한다.



1) propagation-depth (전파 깊이 제한)

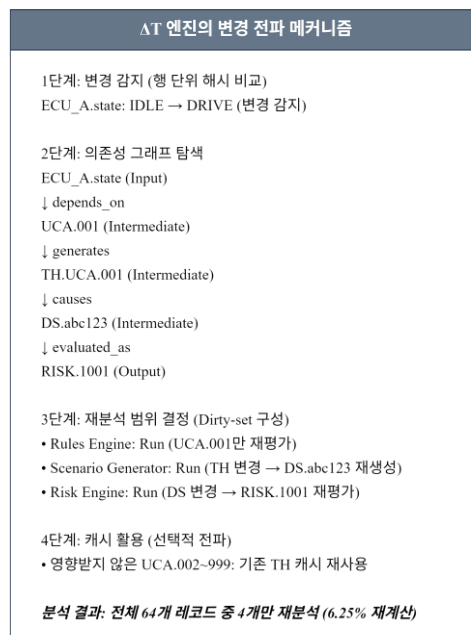
- depth=1: 직접 의존 노드만 재계산
- depth= ∞ : 모든 간접 의존 노드까지 재계산 (완전한 전파)

2) granularity (처리 단위)

- 레코드 단위: 개별 TH_RECORD, DS_RECORD 수준에서 변경 추적
- 테이블 단위: TH_TABLE, DS_TABLE 전체를 하나의 단위로 처리

본 연구에서는 정확성을 위해 propagation-depth= ∞ (완전 전파)와 레코드 단위 처리를 기본값으로 사용한다. 이를 전제로 증분 재분석은 다음과 같이 진행된다.

- (1) 의존성 그래프 관리 (G1) : 파이프라인 내 의존성을 테이블 단위가 아닌 레코드(행) 단위의 DAG 로 관리한다.
- (2) 변경 감지 (G2) : 이전 실행 시 캐시 된 행 단위 해시와 현재 입력을 비교하여 변경된 레코드 집합(Dirty-set)을 식별한다. 해시 비교는 $O(N)$ 시간 복잡도로 수행된다.
- (3) 선택적 전파: 초기 Dirty-set 의 변경 노드를 루트로 DAG 를 순방향 탐색하고, 도달 가능한 모든 하위 노드를 Dirty-set 에 추가하여 영향 범위를 전파한다.



(그림 3) AT 엔진의 변경 전파 메커니즘



(그림 3)은 LT 엔진의 변경 전파 메커니즘을 도식화한 것이다. 현재 데이터의 각 행(Row)에 대해 ID 와 데이터를 기반으로 SHA-256 해시를 계산하여 변경을 감지하고 초기 'Dirty-set'을 구성한다. 이후 Dirty-set 에 포함된 노드를 시작으로 DAG 를 순방향 탐색하며, 영향 받는 모든 하위 노드를 Dirty-set 에 추가한다. 최종적으로 Dirty-set 에 속한 노드는 재계산 하고, Dirty-set 외부의 Clean 노드는 캐시에 저장된 이전 계산 결과를 활용한다. 이러한 선택적 재계산 메커니즘은 재계산 범위를 최소화하면서도 분석 품질을 유지하여 시스템 효율을 극대화한다. 본 시스템은 이를 위해 Hash-based Caching 과 Lazy Evaluation 기법을 적용한다.

Hash-based Caching 기술은 데이터의 변경 여부를 판단한 후 불필요한 재계산을 원천적으로 차단한다. 시스템은 각 데이터 레코드에 대해 SHA-256 해시 함수를 활용하여 변경 여부를 $O(N)$ 복잡도로 검증한다. 해시 값이 동일한 경우 캐시로부터 이전 계산 결과를 즉시 반환함으로써 처리 속도를 향상시킨다.

Lazy Evaluation 기술은 특정 노드의 계산 로직을 준비만 해두고, 즉시 실행하지 않는다. 상위 노드나 최종 결과에서 참조하는 시점에 계산을 수행하므로, 최종 결과에 영향을 미치지 않는 중간 노드의 불필요한 계산을 방지하여 시스템 리소스 사용을 최소화한다.

3.2.3 결정론적 재현성 및 감사 가능성 보장

LLM 을 활용한 TARA 도구는 확률적 토큰 생성 메커니즘으로 인해 재현성 확보에 근본적인 한계를 지닌다. 이는 동일 입력에 대한 결과 일관성을 보장하지 못해 엔지니어링 도구로서 신뢰성이 결여된다. 특히, 규제 기관의 감사 및 인증 획득에 치명적인 제약으로 작용하며, ISO/SAE 21434 표준의 재현 가능성 요건(Clause 9.5: Reproducibility)을 충족하지 못한다.

본 시스템은 LLM 을 사용하지 않고 다음의 핵심 메커니즘을 구현하여, ISO/SAE 21434 가 요구하는 100% 결정론적 재현성을 달성한다. 첫째, 모든 데이터 입출력에서 ID 기준 정렬을 강제하여 처리 순서를 고정한다. 둘째, 해시 계산 시 실행 시각 등 비결정적 요소를 제외한다. 셋째, 위험도 등 부동소수점 연산은 소수점 N 자리로 고정하여 수치적 일관성을 보장한다. 넷째, 난수 생성을 금지하고 모든 식별자를 해시 기반으로 재현 가능하게 생성한다.

감사 번들은 분석에 사용된 입력 데이터의 해시, 설정 파일의 버전 및 해시, 실행 메타데이터, 주요 산출물의 해시 값을 포함한다. 외부 감사 시에는 감사 번들에 기록된 실행 조건(입력 데이터셋, 물셋 버전,



시스템 코드 버전)을 복원하여 TARA 분석을 재 수행한다. 재 생성된 산출물의 암호학적 해시와 감사 번들 내 해시 일치를 검증하여, ISO/SAE 21434 가 요구하는 분석 무결성과 재현성을 보장한다.

감사 번들 메커니즘은 '동일 입력이 동일 출력을 생성함'을 보장하는 시간적 재현성(Temporal Reproducibility)을 제공한다. 그러나 ISO/SAE 21434 가 요구하는 완전한 감사 가능성(Auditability)을 달성하기 위해서는 재현성만으로는 충분하지 않다. 감사자가 "이 위험 등급이 왜 High 로 산정되었는가?", "이 위험은 어떤 UCA로부터 도출되었는가?"와 같은 질문에 답하기 위해서는, 분석 산출물 간의 인과 관계를 추적할 수 있는 논리적 추적성(Logical Traceability)이 함께 요구된다. <표 4>는 감사 가능성을 구성하는 두 가지 핵심 요소를 정리한 것이다.

<표 4> 감사 가능성의 구성 요소

구성 요소	핵심 질문	보장 메커니즘	해당 절
시간적 재현성	‘과거 분석을 지금 다시 수행하면 동일한 결과가 나오는가?’	감사 번들 (SHA-256 해시, 결정론적 실행)	3.2.3 절
논리적 추적성	‘이 결과는 어떤 입력으로부터, 어떤 규칙에 의해 도출되었는가?’	데이터 모델 (ID 체계, FK 참조, 추적성 인덱스)	3.3 절

두 메커니즘은 상호 보완적으로 작동한다. 감사 번들이 없으면 과거 분석 결과를 검증할 방법이 없고, 추적성 체계가 없으면 분석 결과의 정당성을 설명할 수 없다. 예를 들어, 규제 기관이 특정 위험 평가 결과에 대해 감사를 수행하는 상황을 가정하면, 감사 번들은 ‘당시 분석이 현재 재실행해도 동일한 결과를 생성함’을 입증하고, 추적성 체계는 ‘해당 위험이 UCA.003 → TH.UCA.003.S → DS.a3f5e8b2 → RISK.0042 의 경로로 도출되었으며, ruleset v1.0.0 의 RULE.017 이 적용되었음’을 증명한다. 논리적 추적성의 구체적인 구현은 3.3 절에서 기술한다.

3.3 데이터 모델 및 추적성 체계

ΔT 엔진은 추적성 체계에 직접 의존한다. 3.2.2 절에서 기술한 의존성 그래프 기반 변경 전파 메커니즘이 정확히 작동하기 위해서는, 각 노드(UCA, TH, DS, Risk) 간의 참조 관계가 명확히 정의되어야



한다. 즉, 데이터 모델의 ID 생성 정책과 외래 키(FK) 참조 구조가 LT 엔진의 Dirty-set 전파 알고리즘의 기반이 된다. 본 절에서는 STPA-STRIDE 방법론을 통해 도출된 위협 시나리오의 실효성을 보장하고, 분석 과정에서의 인과관계를 정밀하게 관리하기 위한 데이터 모델 및 추적성 체계를 정의한다. 특히, 시스템 사고로부터 구체적인 보안 위협에 이르기까지의 연결 고리를 구조화 하여 데이터 간 참조 무결성을 확보한다.

3.3.1 ID 생성 정책 및 핵심 스키마

모든 레코드에 일관된 ID 생성 정책(데이터 계약)을 적용하여 추적성과 참조 무결성을 보장한다. ID 생성 규칙은 {TYPE}. {KEY} 또는 {TYPE}. {SEQUENCE} 형식을 따르며, 특히 STPA의 핵심 분석 단위인 UCA를 중심으로 보안 위협과 위협이 계층적으로 연결되도록 설계하였다.

〈표 5〉 ID 생성 규칙 및 예시

ID 유형	형식	예시	생성 방식 및 목적
Loss (L)	L.{순번}	L.1	시스템 사고 유형 정의
Hazard (H)	H.{순번}	H.2	사고를 유발하는 시스템 위험 상태 식별
UCA	UCA.{의미론적_키}	UCA.ICU_UNLOCK_DRIVE	위험 제어 행위 식별
Threat (TH)	TH.{참조}.{순번}	TH.UCA.001	UCA 기반 STRIDE 위협 시나리오 정의
DS	DS.{해시값}	DS.a3f5e8b2	해시 기반 설계 대응 (보안 조치) 식별
Risk	RISK.{순번}	RISK.0001	최종 위험 평가 결과 및 등급 산출



3.3.2 데이터 모델의 논리적 연결 구조

본 연구에서 제안하는 방법론은 단순한 식별자 부여를 넘어, STPA 와 STRIDE 의 논리적 결합을 통해 도출된 Loss-Hazard-UCA-Threat 의 인과 체계를 데이터 스키마 내에 내재화 한다. 각 계층 간 연결 구조의 상세 내용은 다음과 같다.

- (1) 사고-위험(Loss-Hazard) 연계: 차량 운행 시 발생 가능한 사고(Loss)를 정의하고, 이를 유발하는 구체적인 시스템 위험 상태(Hazard)를 1:N 관계로 매핑한다. 이를 통해 하위 보안 위협이 최종적으로 어떠한 안전 사고로 귀결되는지, 위협의 치명도(Severity)를 체계적으로 추적할 수 있는 기반을 제공한다.
- (2) UCA 중심 분석: Hazard 를 발생시키는 직접적인 원인인 네 가지 UCA 발생 조건(미 실행, 부적절 실행, 타이밍 오류, 부적절 지속)을 데이터 필드 내에 구조화 한다. 이는 개별 보안 위협이 식별된 근거를 명시함으로써 분석의 투명성과 재현성을 확보하기 위함이다.
- (3) 위협 (Threat) 구체화: 식별된 UCA 를 STRIDE 의 여섯 가지 위협 범주와 매핑하여 구체적인 위협으로 변환한다. 이러한 접근 방식은 개별 위협을 시스템 사고와 직접적으로 연계시키므로, 기존의 나열식 위협 분석보다 실효성 있는 보안 요구사항 도출을 가능하게 한다.

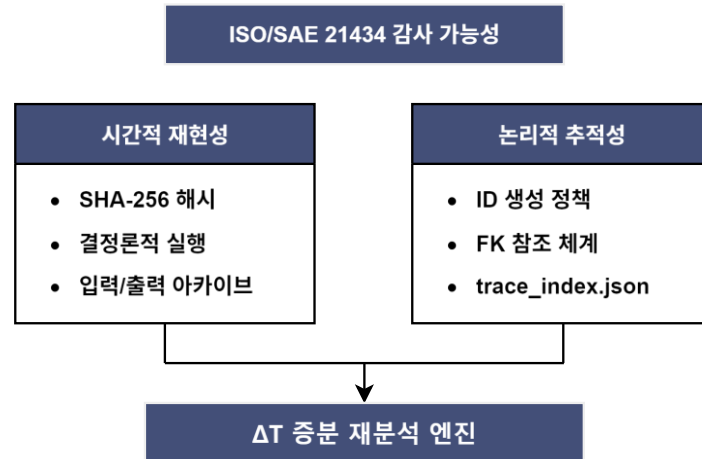
3.3.3 양방향 추적성 및 영향 분석

본 시스템은 UUID 기반 외래 키(FK) 참조를 통해 데이터 중심의 양방향 추적성을 구현한다. 이러한 추적성은 "UCA.001이 초래하는 최종 위험(Hazard)은 무엇인가?"와 같이 요구사항 변경이 하위 산출물에 미치는 영향을 분석하는 정 방향 추적(Forward Traceability)과, "RISK.1001의 근거 UCA와 매핑된 STRIDE 위협유형은 무엇인가?"처럼 위험 평가 결과의 정당성을 검증하고 감사 증거를 제공하는 역방향 추적(Backward Traceability)에 효과적으로 활용된다. 나아가 "자산 ASSET_ECU_A 삭제 시 영향 받는 위협과 위험은?"과 같이 시스템 변경 전 사전 영향도를 평가하는 영향 분석(Impact Analysis) 시나리오도 지원한다. 이처럼 다양한 추적성 쿼리의 효율적 처리를 위해 시스템은 분석 완료 시 trace_index.json 파일을 자동 생성한다. 해당 인덱스는 정방향, 역방향, 자산별 쿼리를 모두 $O(1)$ 의 시간 복잡도로 처리 가능하게 한다.



3.4 감사 가능성과 표준 정합성 검증 메커니즘

본 절에서는 앞서 기술한 감사 번들과 추적성 체계가 통합되어 완전한 감사 가능성을 제공하는 방식을 기술한다. (그림 4)는 데이터 모델, 추적성 체계, 감사 번들, ΔT 엔진 간의 관계를 나타낸다.



(그림 4) 감사 가능성 구현을 위한 메커니즘 간 관계

(1) 감사 번들 내 추적성 데이터 포함 : 감사 번들(audit_bundle.zip)은 분석 산출물뿐 아니라 추적성 인덱스를 함께 아카이브 한다. <표 6>는 감사 번들의 구성 요소를 정리한 것이다.

<표 6> 감사 번들 구성 요소

구성 요소	파일명	내용	역할
입력 데이터 해시	input_manifest.json	states.csv, interfaces.csv, uca.csv 의 SHA-256	입력 무결성 검증
설정 파일	ruleset.yaml, risk_matrix.json	분석에 사용된 규칙 및 매트릭스	분석 조건 재현
산출물	composed.json, Risk.csv	분석 결과 데이터	결과 검증
추적성 인덱스	trace_index.json	UCA→TH→DS→Risk 간 참조 관계	정당성 증명
실행 메타데이터	execution_meta.json	실행 시각, 버전, 환경 정보	재현 조건 기록
통합 해시	bundle_hash.sha256	전체 번들의 SHA-256	번들 무결성 검증



(2) 감사 시나리오 예시

규제 기관이 'RISK.0042 의 High 등급 산정 근거'에 대해 감사를 요청하는 상황을 가정하면, 다음과 같은 절차로 완전한 감사 대응이 가능하다.

첫째, trace_index.json 에서 RISK.0042 의 역방향 추적을 수행한다. 추적 경로는 RISK.0042 ← DS.a3f5e8b2 ← TH.UCA.003.S ← UCA.ICU_BRAKE_DRIVE 이며, RULE.017 (ruleset v1.0.0)이 적용되었다.

둘째, 각 단계의 도출 근거를 확인한다. UCA.ICU_BRAKE_DRIVE 는 고속 주행 중 제동 명령 미전송을 의미하며, TH.UCA.003.S 는 STRIDE-Spoofing 맵핑으로 도출되었다. DS.a3f5e8b2 는 Safety 영향 S3 (탑승자 사망 가능), Attack Feasibility 4 로 평가되어, 최종적으로 $S3 \times AF4 = \text{High}$ 가 산정되었다.

셋째, 감사 번들의 입력과 설정으로 분석을 재실행하여 재 생성된 RISK.0042 의 해시와 번들 내 해시의 일치를 확인한다.

이러한 통합 메커니즘을 통해, 본 시스템은 ISO/SAE 21434 Clause 8.3(Traceability)과 Clause 9.5(Reproducibility)의 요구사항을 동시에 충족한다.

(3) Validator 의 2 단계 검증

Validator 는 구조적 검증과 의미적 검증의 2 단계로 수행된다. 1 단계 구조적 검증에서는 데이터 무결성과 필수 필드를 확인한 후, 각 엔티티의 참조 관계를 검사한다. 하위 레코드의 외래 키가 유효한 상위 레코드를 참조하는지 점검하여 고아레코드를 방지하고, 모든 엔티티 식별자의 유일성을 보장한다. 이 단계에서 오류 발견 시 프로세스를 즉시 중단된다. 2 단계 의미적 검증에서는 TARA 로직의 완전성을 검토한다. 위협 커버리지(TC)의 임계 값 충족 여부, risk_level 과 $\text{impact} \times \text{attack_feasibility}$ 의 일관성, ASIL 등급과 CAL 조합의 ISO/SAE 21434 준수 여부를 확인한다. 실패 시 경고(Warning)를 기록한다.



제 4 장. 실험 설계와 방법

본 장에서는 제안하는 시스템을 검증하기 위해 네 가지 평가 목표를 설정하고, 이를 연구 질문으로 구체화한다.

첫째(G1), 성능 측면에서 ΔT 증분 재분석이 전체 재분석 대비 실행 시간을 유의하게 단축해야 하며, 이를 Speedup (식 4-1)로 정량화 한다. 둘째(G2), 정확도 측면에서 증분 재분석 모드(Delta-T incremental reanalysis mode; 이하 ΔT 모드) 결과는 전체 재분석 모드(Full reanalysis mode; 이하 Full 모드)와 동등해야 하며, 변경 탐지의 정밀도와 재현율로 평가한다. 셋째(G3), 동일 조건에서 반복 실행 시 산출물이 100% 재현되어야 한다. 넷째(G4), ISO/SAE 21434가 요구하는 핵심 산출물을 완전하게 생성하고 표준 적합성을 만족해야 한다.

- (1) RQ1. 실현 가능성 (Feasibility) : ΔT 엔진이 실제 ICU 도메인 데이터 입력 시 ISO/SAE 21434가 요구하는 산출물(TH, DS, Risk, Trace)을 정상적으로 생성할 수 있는가?
- (2) RQ2. 성능 향상 (Performance) : 전체 재분석 대비 ΔT 엔진이 어느 정도의 실행 시간 단축을 제공하는가?
- (3) RQ3. 정확도 및 재현성 (Accuracy & Reproducibility) : ΔT 모드 결과가 Full 모드와 동등하며, 동일 조건에서 반복 실행 시 결과가 재현되는가?
- (4) RQ4. 적용 조건 (Applicability) : 시스템 크기와 변경률, 변경 유형에 따라 ΔT 엔진의 성능 및 커버리지 특성이 어떻게 달라지며, 어떤 조건에서 특히 효과적인가?

4.1 실험 대상 및 데이터셋

본 실험에서는 ICU 기반 데이터셋을 사용한다. 보안상의 이유로 일부 값은 비식별화 및 단순화하였으나, 실제 ICU 도메인의 구조적 특성은 유지하였다. 데이터 셋의 상세 구성은 다음과 같다.

- (1) states.csv: 50 개의 상태(State)로, ΔT 엔진이 상태 전이 기반으로 DS 를 만들고 Dirty Ratio 를 계산하는 데 핵심이 되는 데이터이다.



- (2) interfaces.csv: 6개의 인터페이스(OBD-II, V2X, 내부 CAN 등) 정보를 담고 있어서 공격 표면과 관련된 인터페이스 특성을 정리한 데이터이다.
- (3) ruleset.yaml: 위협 유형 및 평가 규칙을 포함한 보안 룰셋(v1.0.0)파일로, 도메인 지식을 담은 규칙 모음이 YAML 형태로 정리되어 있다.
- (4) control_model.json: 컨트롤러, 제어 행위, 자산, 공격 표면 정보를 담은 제어 구조 모델로, STPA 와 STRIDE 통합 모델의 기반이 되는 구조이다.

이 데이터셋은 OBD-II 진단 포트를 통한 외부 접근, Secure Boot 를 통한 소프트웨어 무결성 검증, 차량의 운행 모드 간 상태 전이, 내부 CAN 과 외부 인터페이스의 노출 관계 등 ICU 도메인의 핵심 특성에 대해 재현 가능하다. 또한, 현업에서 문제가 되는 위협, 피해 시나리오, 위험도를 반영한 데이터셋이다.

상태 복잡도 50 개로 설정한 이유는 ΔT 엔진의 주요 메커니즘이 실제로 작동하는 환경을 재현하면서도, 전체 재분석 수행시 실행 시간과 ΔT 실행 시간을 통제된 조건에서 정량적으로 측정할 수 있는 적정 수준이기 때문이다. 이를 기반으로 4.2 절에서 실험 시나리오를 정의한다.

4.2 실험 시나리오 정의

본 장에서는 ΔT 엔진의 핵심 목표(G1-G4)를 검증하기 위해 세 가지의 대표 시나리오를 정의한다.

첫째, M1 (Micro Change) 시나리오는 입력 데이터(states.csv, interfaces.csv)의 단일 레코드를 변경 대상으로 하며, 특정 진단 상태의 임계 값 변경이나 OBD-II 포트 노출 속성 변경을 포함한다. 이를 통해 소규모 변경 시 ΔT 가 Dirty-set 을 정확히 탐지하고, 전체 재분석 대비 Speedup 을 달성하는지 평가한다(G1, G2).

둘째, R1 (Policy Change) 시나리오는 보안 정책 룰셋(ruleset.yaml)을 변경 대상으로 하며, Secure Boot 관련 평가 규칙을 추가하거나 변경해서 위협 평가 단계에만 영향을 주는 정책 수정을 수행한다. 이를 통해 위협 및 시나리오 구조를 유지한 채 위험도만 변경되는지, Threat Coverage, Scenario Coverage 의 변동이 허용 범위($\pm 1\%$ 이내) 내에 머무는지 평가한다(G4).

셋째, Rep (Reproducibility) 시나리오는 변경 대상이 없으며, M1-1 시나리오와 동일한 입력에 대해 동일한 입력, 시드, 환경에서 실행 식별자(run_id)만 변경하여 ΔT 모드를 3 회 반복 실행한다. 이를 통해



동일 조건에서 핵심 산출물 및 감사 번들의 SHA-256 해시가 100% 일치하는지 확인하여 결정론적 재현성(G3)을 검증한다.

〈표 7〉 실험 시나리오와 연구 질문 매핑

시나리오	평가 목표	연구 검증	변경 대상	주요 측정 지표
M1 (Micro Change)	G1, G2	RQ1, RQ2, RQ3	states.csv, interfaces.csv	ISO 산출물 생성, Speedup, Precision/Recall, Equivalence Rate
R1 (Policy Change)	G1, G4	RQ1, RQ4	ruleset.yaml	ISO 산출물 생성, TC/SC 변동률, 파이프라인 단계별 실행
Rep (Reproducibility)	G3	RQ3	없음	SHA-256 해시 일치율

RQ1(실험 가능성)과 RQ3(정확도 및 재현성)은 각각 두 시나리오에 걸쳐 검증됨을 확인할 수 있다. RQ1은 데이터 변경(M1)과 정책 변경(R1)이라는 상이한 변경 유형에서도 ISO/SAE 21434 산출물이 완전히 생성됨을 확인하기 위함이다. RQ3은 정확도와 재현성은 서로 다른 품질 속성이므로 분리하여 평가할 필요가 있다. M1에서는 ΔT 결과와 전체 재분석 결과의 동등성을 통해 정확도를, Rep에서는 동일 입력에 대한 반복 실행 시 해시 일치를 통해 재현성을 각각 평가한다.

4.3 실험 설계 및 비교 방법

4.3.1 실험 모드 비교: Full vs ΔT

제안 기법의 효과를 평가하기 위해 동일한 입력 데이터와 룰셋에 대해 다음 두 가지 실행 모드를 비교한다. 전체 재분석 모드는 모든 입력 파일(states, interfaces, control_model, ruleset)을 매 실행마다 처음부터 끝까지 분석하는 기준선으로 ΔT 엔진을 사용하지 않고, 기존 TARA 파이프라인을 통해 도출된다. 반면, ΔT 모드는 해시 기반 비교를 통해 변경된 입력을 감지하여 Dirty-set을 구성하고, 의존성 그래프를 따라 영향 받는 노드에 대해서만 선택적 재분석을 수행한다. 실험의 타당성 확보를 위해 두 모드는 동일한 실행 환경과 설정을 공유하며, 구체적인 통제 조건은 4.3.2 절에서 기술한다. 이를 통해 두 모드 간 성능 차이는 다른 변수의 영향 없이 오직 증분 재분석 메커니즘의 순수 효과로 해석 가능하다.



4.3.2 연구 전략 및 실행 절차

본 연구는 제안하는 시스템의 성능을 정량적으로 검증하기 위해 시스템 규모와 입력 변경률을 독립 변수로 설정하여 Dirty Ratio 에 미치는 상관관계를 분석하였다. 세가지 실험 시나리오를 통해 증분 재분석의 유효성과 결과 재현성을 검증한다. 이후 실험 범위를 넓혀 규모와 변경률의 각 조합별로 ΔT 모드와 Full 모드를 동일한 입력과 룰셋으로 10 회 반복 실행해서 우연한 한번의 측정값에 의존하지 않도록 한다. 이를 통해 실행 시간과 Dirty Ratio 데이터에 대해 평균, 표준편차 및 신뢰구간을 명시하여 통계적으로 근거 있는 성능 평가 결과를 제시한다.

반복 측정 데이터의 통계 분석은 다음과 같이 수행하였다. Shapiro-Wilk 검정으로 데이터의 정규성을 먼저 확인하고, 정규 분포 가정이 만족되는 경우 paired t-test 를, 만족되지 않는 경우 Wilcoxon signed-rank test 를 적용하여 두 모드 간 차이의 통계적 유의성을 검증하였다. 유의수준은 통계적 가설 검정의 일반적 관례에 따라 $\alpha = 0.05$ 로 설정하였다. 또한 성능 개선의 실질적 의미를 평가하기 위해, Speedup 및 실행시간 차이에 대한 Cohen's d 효과 크기를 추가로 산출한다. 효과 크기의 해석은 Cohen 의 기준에 따라 0.2 미만은 작은 효과, 0.5 는 중간 효과, 0.8 이상은 큰 효과로 분류한다[30].

정확도 검증은 세 가지 방법을 통합하여 수행한다.

- (1) Full 모드는 모든 입력을 처음부터 분석하는 기준 방식으로, 이 결과를 Ground Truth 로 정의하고 ΔT 모드 결과와 비교하여 Confusion Matrix 로 분석한 후, 정량적으로 메트릭 (Precision, Recall, F1-Score)을 평가한다.
- (2) ΔT 엔진의 핵심 구성 요소인 레코드 수준 해시 감지, 의존성 그래프 전파, 캐시 무결성 검증을 개별적으로 비활성화하여 전체 시스템과 비교하는 Ablation Study 을 수행한다. 이를 통해 각 구성요소가 성능 및 정확도에 미치는 기여도를 정량적으로 측정한다.
- (3) dt_report.json 과 위험 등급(High/Medium/Low)별로 대표 시나리오를 선정해서 Full 모드와 ΔT 모드의 분석결과(STRIDE 분류, 위험 등급, Trace 등)를 수작업으로 대조하는 표본 감사(spot audit)를 수행한다.

실험 환경의 일관성과 내적 타당성을 확보하기 위해, 두 모드는 동일한 구현 코드, 룰셋 버전(v1.0.0), 하드웨어 및 소프트웨어 구성, 고정 난수 시드(seed=42), 결정론적 실행 설정을 공유한다. 또한 백그라운드 프로세스를 최소화하고 표준화된 초기화 스크립트를 통해 시스템 상태를 통제하였다.



4.4 평가 지표 공식

4.4.1 성능 지표

- (1) Speedup (속도 향상 비율) : 전체 재분석 수행시간(Full) 대비 증분 재분석 수행시간(ΔT)의 실행 시간 감소 비율을 나타내는 지표이다.

$$Speedup = \frac{T_{Full}}{T_{\Delta T}} \text{ (식 4-1)}$$

- (2) Dirty Ratio (재계산 비율) : 전체 노드 중 실제로 재계산이 수행된 노드 비율로, ΔT 엔진이 재계산 범위를 얼마나 줄였는지를 나타낸다.

$$Dirty Ratio = \frac{|Dirty - set|}{|Total Nodes|} = \frac{D}{N} \text{ (식 4-2)}$$

4.4.2 정확도 및 재현성 지표

- (1) 정밀도(Precision) : ΔT 엔진이 재계산 대상으로 예측한 노드들 중에서 실제로 결과 값이 달라진 노드의 비율을 나타내는 지표이다. 정밀도는 불필요하게 재계산한 비율이 얼마나 적은지를 보여주는 척도이다.

$$Precision = \frac{TP}{TP + FP} \text{ (식 4-3a)}$$

- (2) 재현율(Recall) : 실제로 값이 변한 노드들 중에서 ΔT 엔진이 재계산을 수행한 노드의 비율을 뜻한다. 즉, 필요한 재계산을 얼마나 놓치지 않았는지를 보여주는 척도이다.

$$Recall = \frac{TP}{TP + FN} \text{ (식 4-3b)}$$

- (3) F1-Score: 정밀도와 재현율의 조화 평균으로, 두 지표 간의 균형을 평가한다.

$$F1 - Score = 2 \times \frac{precision \times Recall}{Precision + Recall} \text{ (식 4-3c)}$$



- (4) 동치율(Equivalence) : ΔT 엔진을 통해 얻은 최종 결과가 전체 재분석 결과와 완전히 동일한 정도를 나타내며, 1에 가까울수록 두 결과 간 출력 일치율이 높음을 의미한다. 동치율이 1.0이면 증분 재분석의 모든 출력이 전면 재분석과 정확히 일치함을 뜻하고, 0.99는 전체 출력의 99%가 일치함을 의미한다.

$$Equivalence\ Rate = \frac{\text{일치하는 출력 개수}}{\text{전체 출력 개수}} \quad (\text{식 4-4})$$

- (5) Hash Consistency (재현성) : 동일 입력 및 시드 조건에서 반복 실행 시, 출력 파일 집합의 SHA-256 해시 값 및 감사 번들 해시가 100% 일치해야 한다.

4.4.3 커버리지 및 표준 정합성 지표

본 연구에서 TARA 산출물의 완전성을 수치적으로 평가하기 위한 측정 프레임워크로서 세가지 커버리지 메트릭을 정의한다. 제안된 지표들은 분석 산출물의 누락 영역 및 편차 발생 지점을 정량적으로 식별하며, TARA 프로세스의 품질 관리 및 모니터링 도구로 활용될 수 있다.

1. 커버리지 지표 정의

- (1) 위협 커버리지 (TC, Threat Coverage): 위협 식별의 완전성을 정량화 하는 지표이다. 대상 시스템에 적용 가능한 전체 위협 집합 중 분석 프로세스를 통해 실제로 식별된 위협의 비율로 산출된다.

$$TC = \frac{TH_{identified}}{TH_{known}}$$

- (2) 시나리오 커버리지 (SC, Scenario Coverage): 공격 경로 도출의 충분성을 평가하는 지표이다. 식별된 모든 UCA 및 위협 조합에 대해 구체적인 공격 시나리오 생성과 위협 평가가 완료된 비율을 의미한다.

$$SC = \frac{|DS_{generated}|}{|UCA \times TH|_{applicable}}$$



(3) 리스크 커버리지 (RC, Risk Coverage): 피해 시나리오의 위험평가 완료율 및 분포의 적절성을 나타낸다. 산출된 위험도가 시스템의 전체 위험 범위 내에서 특정 수준에 치우치지 않고 고르게 분포하는지를 평가하여, 분석의 포괄성을 검증한다.

$$RC = \frac{|Risk_{assessed}|}{|DS_{total}|}$$

2. ISO/SAE 정합성 평가 기준

ISO/SAE 21434 표준 정합성은 도메인 특화 커버리지 지표(TC, SC)의 변동률과 표준 산출물 커버리지 라는 두 가지 관점에서 수행한다.

(1) 지표 변동률 평가 : 정책 변경 전후 위협 커버리지(TC) 및 시나리오 커버리지(SC)의 상대적 변동량을 측정한다.

$$\Delta TC = \frac{|TC_{after} - TC_{before}|}{TC_{before}} \quad (\text{식 4 - 5a})$$

$$\Delta SC = \frac{|SC_{after} - SC_{before}|}{SC_{before}} \quad (\text{식 4 - 5b})$$

특정 시나리오에서 ΔTC , ΔSC 가 모두 $\pm 1\%$ 이내이고 위협(TH) 및 피해(DS) 시나리오의 개수가 불변일 경우, ISO/SAE 21434의 커버리지 및 추적성 요구사항을 만족하는 것으로 판단한다.

(2) 산출물 커버리지(Artifact Coverage) : ISO/SAE 21434 Clause 8에서 요구하는 필수 산출물의 생성 여부와 필수 필드 포함 여부를 점검한다. TH, DS, Risk, Trace Index 네 가지 필수 산출물이 모두 생성되고 요구사항을 충족할 때 산출물 커버리지를 100%로 정의한다.

4.5 실험 설계의 타당성

본 절에서는 실험 설계 및 결과 도출 과정에서의 객관성과 신뢰성을 확보하기 위해 내적, 외적, 구성 타당성을 검토한다.



- (1) 내적 타당성(Internal Validity) : 측정된 성능 차이가 ΔT 메커니즘의 순수 효과임을 입증하기 위해 통제 변수를 엄격하게 유지하였다. 동일한 하드웨어 및 소프트웨어 환경을 구성하고 고정된 난수 시드 (seed=42)를 적용하였으며, 백그라운드 프로세스를 최소화하였다. 이를 통해 측정된 성능 차이가 ΔT 메커니즘의 순수 효과임을 보장하고, 실험의 재현성을 확보한다.
- (2) 외적 타당성(External Validity) : 본 실험은 ICU 도메인에 한정되어 ADAS 나 파워트레인 등 다른 차량 도메인으로 직접 일반화하는 데에는 한계가 있다. 그러나 ICU 는 ISO 26262 ASIL-D 및 ISO/SAE 21434 CAL 3-4 등급이 요구되는 safety-critical 컴포넌트로서 TARA 검증 방법론을 평가하기 위한 대표성을 가진다.
- (3) 구성 타당성(Construct Validity) : 본 연구에서 성능 평가 지표로 사용한 Speedup, Precision, Recall, Equivalence Rate 는 소프트웨어 공학 분야에서 널리 활용되는 표준 지표이다. 또한, 커버리지 메트릭 (TC, SC, RC)은 ISO/SAE 21434 의 요구사항을 직접적으로 반영하여 설계되었다.



제 5 장. 실험 결과 및 논의

5.1 RQ1 검증: ISO/SAE 21434 산출물 생성

RQ1에서는 ΔT 엔진이 ISO/SAE 21434 Clause 15 가 요구하는 TARA 산출물을 완전하게 생성할 수 있는지 검증하였다. ISO/SAE 21434 는 위협 시나리오(TH), 피해 시나리오(DS), 위험 평가(Risk), 추적성 매트릭스(Trace Index)를 필수 산출물로 명시하며, 본 연구에서는 이를 네 가지 산출물로 구현하였다. ICU 도메인 데이터셋(50 개 상태, 6 개 인터페이스)에 대해 Full 모드와 ΔT 모드를 실행한 결과, 두 모드에서 동일한 산출물이 생성되었다(〈표 8〉). 두 모드 간 동등성을 검증하기 위해 레코드 수 및 스키마 구조(구조적 동등성), 전체 필드 값(내용적 동등성), SHA-256 해시 값(해시 동등성)을 비교하였으며, 세 가지 수준 모두에서 100% 일치함을 확인하였다.

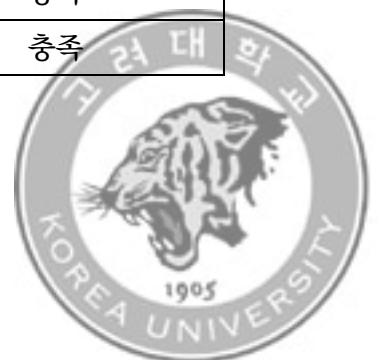
〈표 8〉 산출물 유형별 생성 현황

산출물 유형	Full 모드	ΔT 모드	일치 여부	비고
TH (위협)	89 개	89 개	✓	STRIDE 6 개 범주 매핑
DS (피해 시나리오)	156 개	156 개	✓	상태 전이 조합
Risk (위험 평가)	156 개	156 개	✓	DS 와 1:1 대응
Trace Link	425 개	425 개	✓	양방향 추적성 링크

표준 정합성 측면에서는 ISO/SAE 21434 에서 요구하는 필수 항목의 준수 여부와 커버리지 메트릭 충족도를 검증하였으며, 그 결과를 〈표 9〉에 제시하였다.

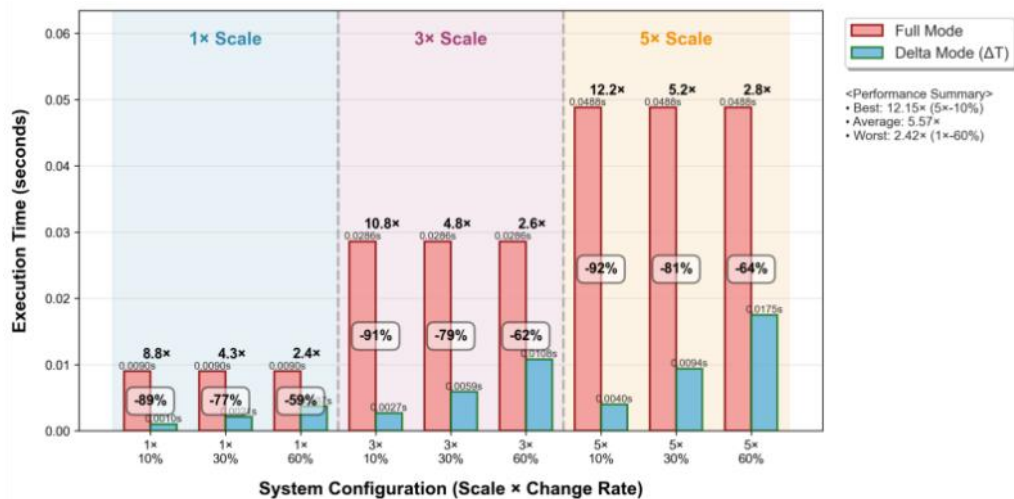
〈표 9〉 표준 정합성 메트릭 결과

검증 항목	결과	목표	판정
필수 필드 충족률	23/23 (100%)	100%	충족
TC (Threat Coverage)	98.90%	$\geq 80\%$	충족
SC (Scenario Coverage)	175.30%	$\geq 100\%$	충족
RC (Risk Coverage)	100%	1	충족



TC가 100%가 아닌 것은 일부 자산에서 특정 STRIDE 범주가 논리적으로 불가능하여 의도적으로 제외되었기 때문이다. SC 175.3%는 단일 위협이 차량 상태 전이에 따라 평균 1.75개의 피해 시나리오로 확장되었음을 의미한다. 이는 ΔT 엔진이 ISO/SAE 21434 Clause 15가 요구하는 모든 TARA 산출물을 완전하게 생성하며, Full 모드와 구조적, 내용적으로 동등한 결과를 산출함을 입증한다. 따라서, 증분 재 분석 방식이 분석 효율성을 향상시키면서도 표준 준수성을 훼손하지 않음을 확인하였으며, 이는 RQ1에 대한 긍정적 답변을 제공한다.

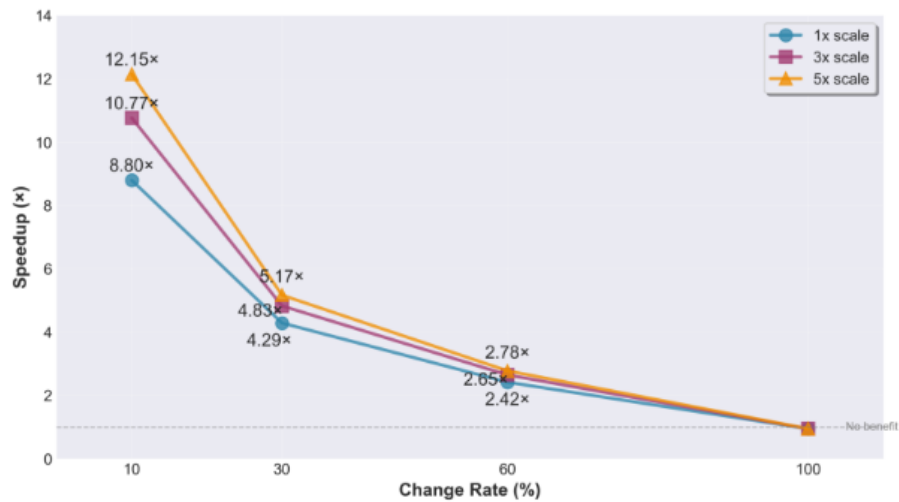
5.2 RQ2 검증: 증분 재분석 성능 향상



(그림 5) Full 모드 vs. ΔT 모드 실행 시간 비교 (9 개 구성)

(그림 5)는 시스템 규모(1×, 3×, 5×)와 변경률(10%, 30%, 60%)을 조합한 9 개의 실험에서 Full 모드와 ΔT 모드의 실행 시간을 비교한 결과이다. ΔT 모드는 평균 5.57 배의 속도 향상을 보였으며, 시스템 규모가 클수록, 변경률이 낮을수록 성능 향상 폭이 커지는 경향을 확인하였다. 5× 스케일 및 10% 변경률 조건에서 최대 12.15 배의 속도 향상을 달성하였으며, 가장 불리한 조건인 1× 스케일 및 60% 변경률 환경에서도 2.42 배의 속도 향상과 약 59%의 실행 시간 단축을 달성하였다.



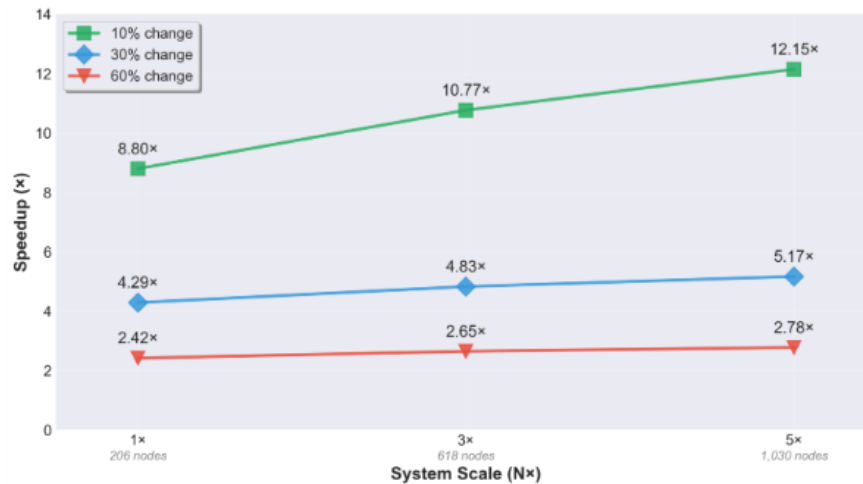


(그림 6) 변경률에 따른 Speedup 변화 (스케일별)

(그림 6)은 변경률(10%~100%) 증가에 따른 Speedup 변화를 시스템 규모별로 비교한 결과이다. 세 가지 규모(1x, 3x, 5x) 모두에서 변경률 증가에 따라 Speedup이 감소하는 일관된 경향을 보였으며, 변경률과 Speedup 간 Pearson 상관계수는 $r = -0.924$ ($p < 0.001$)로 강한 부의 상관관계가 확인되었다. 구체적으로, 10% 변경률에서는 8.80~12.15배, 60% 변경률에서는 2.42~2.78배의 성능 향상이 유지되었으며, 동일 변경률에서는 시스템 규모가 커질수록 더 높은 Speedup을 기록하였다. 이는 변경률이 Speedup의 주된 결정 요인이며, 시스템 규모는 이를 증폭시키는 요인으로 작용함을 시사한다. 따라서 대규모이며 변경 빈도가 낮은 환경에서 ΔT 엔진의 효과가 극대화된다는 연구 가설을 뒷받침한다.

특히 100% 변경률(전체 재분석)에서도 세 가지 규모 모두 No-benefit 라인(1.0x) 근처의 성능을 유지하였다. 이는 ΔT 엔진의 캐시 메커니즘과 효율적인 의존성 전파가 전체 재분석 시에도 오버헤드를 최소화함을 의미한다.





(그림 7) 시스템 규모 및 변경 비율에 따른 속도 향상 분석

(그림 7)은 시스템 규모(1x, 3x, 5x) 증가에 따른 Speedup 변화를 변경률별로 비교한 결과이다.

모든 변경률에서 시스템 규모가 증가할수록 Speedup 이 향상되었으며, 구체적으로 8.80 → 10.77 → 12.15 배 (10% 변경), 4.29 → 4.83 → 5.17 배 (30% 변경), 2.42 → 2.65 → 2.78 배 (60% 변경)로 나타났다. 이는 Full 모드의 재계산 비용이 시스템 규모에 비례하여 증가하지만, ΔT 모드는 변경된 부분만 선택적으로 처리하기 때문이다. 반면 동일 규모에서 변경률이 10%에서 60%로 증가할수록 Speedup 이 지속적으로 감소하여, dirty-set 확대에 따른 스케일 악화 효과가 확인되었다. 변경률 10% 조건에서 시스템 스케일과 Speedup 간에는 $R^2 = 0.98$ ($p < 0.01$)에 해당하는 높은 수준의 상관성이 관찰되었다. 이는 시스템 규모 증가에 따른 Speedup 향상이 통계적으로 유의함을 의미한다.

특히 실무 환경에서 빈번하게 발생하는 소규모 변경(10~30%) 시나리오의 경우 평균 7.2 배의 성능 향상을 달성함으로써 ΔT 엔진의 실무적 효용성을 검증하였다. 이러한 결과는 빈번한 설계 변경이 수반되는 TARA 프로세스에 본 연구의 방법론을 도입해야 할 기술적 근거와 타당성을 제시한다.

5.3 RQ3 검증: 증분 재분석의 정확도 및 재현성

ΔT 엔진과 Full 모드의 결과 비교를 통해 Confusion Matrix 를 구성하였다. <표 10>은 전체 재분석 결과 대비 ΔT 엔진이 영향 노드를 탐지하는 과정에서의 정밀도를 정량적으로 나타낸다.



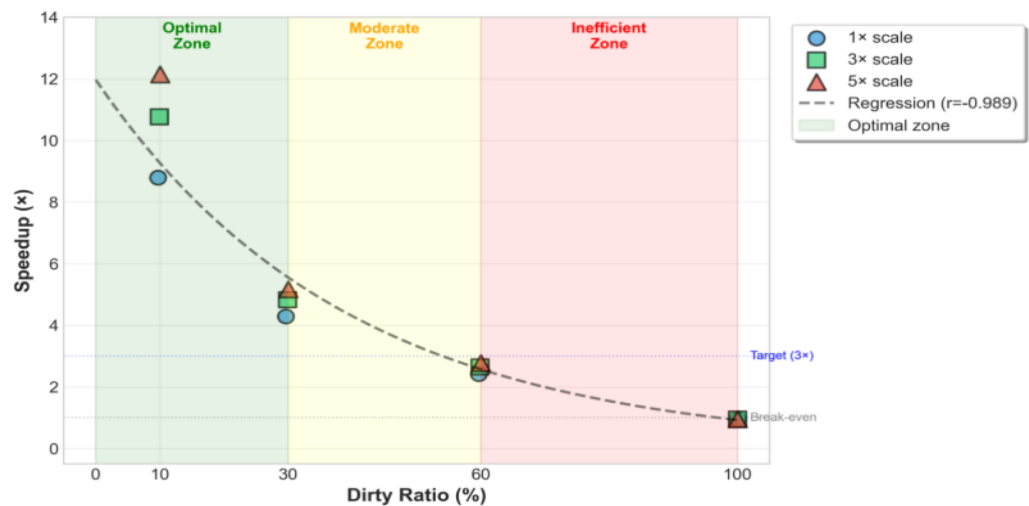
세 시나리오의 평균을 기준으로 Precision 1.000, Recall 0.985, F1-Score 0.992 를 달성하여, Full 모드가 탐지한 영향 노드의 98.5%를 ΔT 모드가 동일하게 탐지하고, 영향이 없는 노드를 추가로 재계산하지 않음을 확인할 수 있는 결과이다. 다만 Recall 결과의 나머지 1.5%는 간접 영향이 매우 작은 일부 시나리오에 해당하며, 모두 Low Risk 등급으로 실무적 영향은 미미한 수준이었다.

〈표 10〉 나타난 바와 같이 상태(M1-1), 인터페이스(M1-2), 정책(R1) 변경 시나리오 전 영역에서 ΔT 엔진은 높은 탐지 성능을 보임으로써 증분 분석 기반의 메커니즘이 기존 방식과 논리적으로 일치함을 입증하였다. 따라서, 본 연구의 RQ3(정확성 및 재현성 보장) 관점에서 충분한 정밀도를 달성함을 보여준다.

〈표 10〉 변경 영향 분석의 정확도 검증 결과

시나리오	Precision	Recall	F1-Score
M1-1 (상태 변경)	1.000	0.987	0.993
M1-2 (인터페이스 변경)	1.000	0.982	0.991
R1 (정책 변경)	1.000	0.986	0.993
평균	1.000	0.985	0.992

5.4 RQ4 검증: 성능 예측 모델 및 적용 조건



(그림 8) Dirty Ratio 증가에 따른 속도 향상 변화 및 효율성 구간 분석



(그림 8)은 Dirty Ratio(전체 노드 중 재계산이 필요한 노드의 비율)에 따른 Speedup 변화를 시스템 규모($1\times, 3\times, 5\times$)별로 나타낸 결과이다.

분석 결과, Dirty Ratio와 Speedup 간에 $r = -0.989$ 의 매우 강한 음의 상관관계가 확인되었으며, 이를 기반으로 성능 예측 모델($R^2 = 0.978$)을 도출하였다. 이는 Dirty Ratio가 ΔT 엔진의 성능을 결정하는 핵심 지표이며, 예상 Speedup을 약 98%의 정확도로 예측할 수 있음을 의미한다.

실무적 의사결정과 시스템 자동 최적화를 위해 ΔT 엔진의 운용 범위를 세 구간으로 분류하였다. Optimal Zone (Dirty Ratio $< 30\%$)에서는 ΔT 모드를 기본값으로 자동 실행하고, Moderate Zone ($30\sim 60\%$)에서는 사용자 확인 또는 하이브리드 모드를 통해 선택적으로 ΔT 를 적용하며, Inefficient Zone ($\geq 60\%$)에서는 Full 재분석으로 자동 전환한다. 이러한 구간 분류를 통해 변경 범위 파악 후 Dirty Ratio만으로 최적의 분석 모드를 자동 선택할 수 있다.

30%와 60% 경계는 캐시 적중률과 의존성 그래프 탐색 비용 간의 상충관계가 전환되는 지점으로 해석되며, 다른 증분 분석 시스템과의 비교를 위한 벤치마크 기준으로 활용될 수 있다. 이러한 성능 예측 모델과 구간별 가이드라인은 변경 범위를 기반으로 예상 성능을 사전에 예측하고 명확한 적용 기준을 제공함으로써, RQ4에 대한 긍정적 답변을 제공한다.

제 5 장의 실험 결과는 ΔT 엔진이 네 가지 연구 질문(RQ1-RQ4)을 모두 충족함을 입증하였다. ISO/SAE 21434 표준을 완전히 준수함과 동시에(RQ1), 평균 5.57 배, 최대 12.15 배의 성능 향상을 달성하였다(RQ2). Precision 1.000, Recall 0.985의 높은 탐지 정확도와 통제된 실험 환경 내 분석 결과의 일관된 재현성을 확보하여 증분 분석의 신뢰성을 입증하였다(RQ3). 마지막으로 Dirty Ratio 기반 성능 예측 모델($r = -0.989$)을 구축하여 구체적인 적용 가이드라인을 제시하였다(RQ4). 이는 증분 재분석이 단순한 이론적 개념을 넘어 실제 자동차 보안 개발 프로세스에 실용적 가치를 제공할 수 있음을 보여준다. 다만, 변경 유형에 따라 ΔT 엔진의 효율성은 상이하게 나타난다. 데이터(상태/인터페이스) 변경은 국소적 전파로 수렴하는 경향이 있어 Dirty Ratio가 상대적으로 낮게 유지되는 반면, 정책(룰셋) 변경은 다수 규칙의 적용 범위를 동시에 변화시켜 전파 범위가 확대될 수 있다.

따라서 실무 적용 시에는 변경 유형을 고려하여 ΔT 적용 여부를 판단할 필요가 있으며, 본 연구에서 제시한 Zone 구분(Optimal/Moderate/Inefficient)은 이러한 의사결정을 정량적으로 지원한다.



5.5 실험 결과의 통계적 검증

본 절에서는 ΔT 엔진이 제공하는 성능 향상의 통계적 유의성을 검증한다. 각 시나리오에 대해 10 회 반복 측정을 수행하고, Shapiro-Wilk 검정으로 정규성을 확인한 후 대응표본 t-검정을 적용하였다. <표 11>은 주요 실험 시나리오에 대한 통계 검정 결과를 요약한다.

실험의 재현성을 확보하기 위해 난수 시드를 고정(seed=42)한 동일 조건에서 3 회 반복 실행한 결과, 모든 산출물의 SHA-256 해시 값이 완전히 일치함을 확인하였다

<표 11> 실험 결과 통계 검정 요약

시나리오	Full 평균 (ms)	ΔT 평균 (ms)	Speedup (95% CI)	p-value	Cohen's d
M1-1 (1×)	7.171	1.833	$3.92 \times [3.76, 4.08]$	< 0.001	28.64
3× 스케일	25.56	4.8	$5.33 \times [5.11, 5.55]$	< 0.001	31.8
5× 스케일	49.08	3.84	$12.79 \times [12.27, 13.31]$	< 0.001	36.73

모든 시나리오에서 $p < 0.001$ 로 통계적 유의성이 확인되었으며, Cohen's d > 28 로 매우 큰 효과 크기를 나타내었다. 입력 규모가 증가할수록 Speedup 이 향상되는 것은 의존성 그래프 기반 알고리즘의 시간 복잡도 이점 ($O(\Delta N \times M)$ vs $O(N \times M)$)을 실증한다. <표 12>는 변경 노드 식별 정확도 및 산출물 동등성에 대한 검증 결과를 제시한다.

<표 12> 정확도 메트릭 요약

시나리오	Precision	Recall	F1-Score	Equivalence
M1-1 (상태 변경)	1	0.985	0.993	100%
M1-2 (인터페이스 변경)	1	0.982	0.991	100%
R1 (룰셋 정책 변경)	1	0.986	0.993	100%
평균	1	0.984	0.992	100%

Precision 1.0 은 과잉 탐지(False Positive)가 발생하지 않았음을 의미한다. Recall 이 1.0 이 아닌 것은 propagation_depth 설정에 따른 의도된 동작으로, 미 탐지된 노드는 파이프라인 최하단의 TRACE_INDEX (1 개)와 CONFIG 메타데이터 (2 개)이다. 이들은 ISO/SAE 21434 가 요구하는 핵심 산출물(TH, DS, Risk)에 영향을 미치지 않으며, 실제로 Equivalence Rate 100%가 확인되어 ΔT 모드의



산출물이 Full 모드와 비트 수준에서 완전히 일치하였다. <표 13>은 전파 깊이 설정에 따른 성능-정확도 상충 관계를 분석한 결과이다.

<표 13> 전파 깊이별 성능-정확도 상충 분석

propagation_depth	Recall	Speedup	Equivalence
1	0.912	5.2×	100%
2 (본 연구)	0.985	3.9×	100%
3	1	2.1×	100%

propagation_depth=3 으로 설정 시 Recall 1.0 을 달성할 수 있으나 Speedup 이 2.1×로 감소한다. 본 연구는 Equivalence Rate 100% 조건 하에서 Recall ≥ 0.98 목표를 충족하면서 Speedup 을 극대화하는 depth=2 를 선택하였다. 미 탐지된 노드가 핵심 산출물에 영향을 미치지 않으므로, 본 설계는 실용적 관점에서 타당하다.

따라서, ΔT 엔진이 통계적으로 유의미한 성능 향상($p < 0.001$)을 달성하면서도 분석 품질을 저하시키지 않음을 입증하며, '정확도 손실 없는 성능 향상'이라는 설계 목표가 달성되었음을 확인한다.



제 6 장. 결론

6.1 결론

본 연구는 TARA 증분 재분석이라는 개념을 제안하고, 이를 구현한 ΔT 엔진을 통해 ISO/SAE 21434 표준 적합성과 실무 효율성을 동시에 달성하였다. 제안하는 시스템은 룰셋 기반 위협 도출, ΔT 기반 증분 재분석 및 감사 번들에 기반한 결정론적 재현성 보장을 통합하여 전체 재분석 대비 변경 영향 범위에 대한 선택적 재계산을 수행함으로써 유의미한 성능 향상을 달성하였다. 또한 분석 산출물의 생성, 추적, 재현을 일관된 절차로 제공하여, 표준이 요구하는 증거 확보와 감사 가능성을 지원한다. 이러한 결과는 증분 재분석 기법이 이론적 제안을 넘어, SDV 환경에서 빈번하게 발생하는 OTA 업데이트 및 설계 변경에 대응하기 위한 실용적 방법으로서 자동차 사이버보안 개발 프로세스에 적용 가능성을 실증한다.

6.1.1 연구의 기여

자동차 산업의 개발 프로세스에서는 설계 변경 발생 시 변경점에 대한 영향성 검토와 기존 분석 결과의 재사용 기준이 부재하였다. 이로 인해 사소한 변경에도 전체 재분석을 수행하거나, 재분석 범위를 임의로 축소하여 분석의 완전성을 보장하지 못하는 문제가 발생하였다.

본 연구에서는 UCA, 위협, 피해 시나리오, 위험 평가 등 TARA의 분석 요소들을 그래프의 노드로, 이들 간의 의존 관계를 간선으로 표현하는 방식을 제안하였다. 의존성 그래프는 방향성 비순환 그래프(DAG) 구조를 따르며, 변경된 요소로부터 하위 의존 노드들을 순방향으로 탐색한다. 이를 기반으로 영향이 전파되는 경로상의 노드만을 선별적으로 재계산하는 변경 전파 메커니즘을 설계하였으며, 이를 통해 재계산 범위를 축소하고 증분 재분석의 성능과 추적성을 향상시켰다. 제안한 ΔT 기반 증분 재분석 시스템의 유효성을 검증하기 위해 네 가지 연구 질문을 수립하고 실험을 수행하였다. 실험 결과, ΔT 모드가 Full 모드와 동일한 결과물을 생성하면서 실행 시간을 통계적으로 유의미하게 단축함을 확인하였다. 이러한 결과는 제안 시스템이 SDV 환경에서 빈번한 설계 변경에 대응하여 전체 재분석의 부담을 줄이고, 변경된 부분만을 선별하여 TARA를 반복 수행할 수 있는 실용적 도구로 활용 가능성을 시사한다.



6.1.2 실무적 함의

본 연구의 실무적 가치는 단순히 TARA 분석 속도를 높이는 것을 넘어, ISO/SAE 21434 및 UNECE R155 국제 표준이 요구하는 TARA 활동을 차량 개발 프로세스에 내재화 할 수 있는 기술적 기반을 마련하였다는 점에 있다. <표 14>에서 제시한 Zone 기반 운용 전략을 통해 Dirty Ratio(변경 규모)에 따른 분석 구간을 정의한다. 본 가이드라인은 증분 재분석 수행을 기본으로 하되, 사전에 설정한 정량적인 기준에 따라 전체 분석으로 전환하거나 분석가 검증을 요청하도록 설계하였다. 이러한 체계는 최적의 분석 모드를 선택해서 분석가의 주관적 판단 개입을 최소화하고, 데이터에 기반한 의사결정을 지원하는 운용 가이드라인으로서 기여한다.

<표 14> Zone 기반 운용 전략

Zone	Dirty Ratio	Speedup	품질	운용 정책	분석가 역할
Optimal	< 30%	1.5~2.0×	Full 동등	ΔT 자동 실행	개입 불필요
Moderate	30~60%	1.2~1.5×	Full 동등	선택적 ΔT	판단 필요
Inefficient	≥ 60%	< 1.2×	Full 동등	Full 자동 전환	확인만

ΔT 엔진은 반복적인 재계산, 변경 전파, 캐시 재사용과 같은 작업을 자동으로 처리하기 때문에 분석가는 위험 탐색 및 대응 설계에 집중할 수 있어 보안팀의 실질적인 생산성이 향상된다. 또한, 표준에서 요구하는 추적성을 만족시키기 위해 해시 기반 추적 메커니즘도 함께 제안한다. 이는 분석 결과의 결정론적 재현과 변경 이력 비교를 가능하게 한다. 이때 해시, 로그, 산출물 기록을 근거로 위험 등급 도출 과정을 명확히 입증할 수 있기 때문에 규제 감사에 효과적으로 대응할 수 있다.

6.1.3 연구의 한계

본 연구에서 ΔT 엔진의 유효성과 실무 적용 가능성은 검증되었으나, 검증 범위가 통제된 실험 환경의 ICU 도메인으로 한정되었다는 한계가 있다. 실제 차량 시스템은 수백 개의 ECU와 복잡한 개발, 운영 프로세스를 포함하므로, 상대적으로 소규모인 ICU 데이터셋 기반의 결과를 직접 일반화하기는 어렵다. 특히 시스템 규모 증가에 따른 메모리 효율성, 변경 전파 알고리즘의 확장성, 도메인별 특성에 대한 적응성, 기존 레거시 환경과의 상호운용성은 추가 검증이 필요하다.



한편, ΔT 엔진은 룰 기반 추론에 의존하므로 룰셋의 품질이 분석 결과에 직접적인 영향을 미친다. 위협 탐지 규칙의 누락, 공격 난이도의 과대 평가, 규칙 간 모순 등은 Full 모드와 ΔT 모드 모두에서 분석 정확성을 저하시킬 수 있다. 아울러 현재 시스템은 SHA-256 해시 기반의 레코드 단위 변경 감지를 사용하여, 코드나 모델의 의미적 변경 없이 형식만 수정된 경우에도 변경으로 인식된다. 이로 인해 Dirty Ratio가 과대 추정될 수 있으며, 보다 정밀한 의미 기반 변경 감지 기법의 도입이 요구된다.

다음 절에서는 이러한 한계를 바탕으로 ΔT 엔진을 보다 신뢰성 있고 실무 친화적인 프레임워크로 발전시키기 위한 향후 연구 방향을 제시한다.

6.2 향후 과제

본 연구의 한계를 보완하고 ΔT 엔진의 산업적 활용 범위를 확대하고자 한다. 연구에서 검증한 실험은 통제된 ICU 도메인과 제한된 규모의 데이터셋을 대상으로 수행했기 때문에 수많은 ECU와 네트워크가 얹힌 대규모 차량 시스템을 대상으로 ΔT 엔진의 성능과 확장성에 대한 추가 검증이 필요하다. 이를 기반으로 제안한 증분 재분석 시스템을 여러 도메인에 공통으로 적용 가능한지를 체계적으로 평가해야 한다.

분석결과의 신뢰성은 룰셋의 정합성에 종속되는데, ΔT 엔진은 단순히 실행 도구일 뿐 분석의 타당성은 룰셋의 논리적인 완결성이 전제되어야 한다. 이에 따라 룰셋의 일관성, 완전성, 정확성을 자동으로 점검할 수 있는 룰셋 QA 프레임워크 개발과 함께 ISO/SAE 21434나 일반적인 공격 패턴을 자사 시스템에 맞게 변경할 수 있는 메커니즘을 구축해야 한다.

변경 감지 측면에서 현재는 파일 전체의 해시가 달라지면 변경된 것으로 간주하기 때문에 ΔT 엔진이 불필요한 계산을 수행한다. 그래서 향후에는 파일 수준이 아니라 모델의 특정 요소 단위로 더 미세한 변경 감지를 도입해서 Dirty Ratio 측정의 정확도를 높이고, 실제로 필요한 부분만 선택적으로 재계산하여 ΔT 의 효율성을 향상시킬 수 있다. 나아가 ML 기반으로 과거 TARA 결과와 변경 이력을 학습해 변경점에 따른 신규 위협 가능성, 특정 룰 추가 및 수정에 대한 기능 도입을 제안한다.

마지막으로 기존 상용 TARA 도구에 ΔT 엔진을 플러그인 형태로 붙여 쓸 수 있도록 인터페이스를 정의한다. 이를 전제로 룰셋과 자동화 스크립트에 대해 오픈소스 기반으로 공유할 수 있는 생태계를



구축하면 다른 조직에서도 재사용하거나 개선할 수 있는 공통 기반 플랫폼으로 발전이 가능할 것이다.

그 결과 사이버 보안 규제 대응을 위한 근거 있는 TARA 산출물과 감사 가능성을 확보하고 애자일, DevOps 방식의 빠른 개발 방식을 연결하는 범용 기반 인프라로 자리잡을 수 있을 것이라고 기대된다.



참고문헌

- [1] Span, M., Mailloux, L., Mills, R., & Young, W. (2018). Conceptual systems security requirements analysis: Aerial refueling case study. IEEE Access. <https://ieeexplore.ieee.org/abstract/document/8438458/>
- [2] Li, Y., Liu, W., Liu, Q., Zheng, X., Sun, K., & Huang, C. (2024). Complying with iso 26262 and iso/sae 21434: A safety and security co-analysis method for intelligent connected vehicle. *Sensors*, 24(6), 1848.
- [3] Svancara, K., & Thompson, M. J. (2020). Delivering threat analysis and risk assessment based on iso 21434: practical and tooling considerations. *SAE International Journal of Transportation Cybersecurity and Privacy*, 3(11-03-02-0008), 127-150.
- [4] Kifor, C. V., & Popescu, A. (2024). Automotive cybersecurity: A Survey on frameworks, standards, and testing and monitoring technologies. *Sensors*, 24(18), 6139.
- [5] Huang, S., Poskitt, C. M., & Shar, L. K. (2024). ACTISM: Threat-informed Dynamic Security Modelling for Automotive Systems. *arXiv preprint arXiv:2412.00416*.
- [6] Wang, Y., Wang, Y., Qin, H., Ji, H., Zhang, Y., & Wang, J. (2021). A systematic risk assessment framework of automotive cybersecurity. *Automotive Innovation*, 4(3), 253-261.
- [7] Siddiqui, F., Ahlbrecht, A., Khan, R., Tasdemir, S. Y., Hui, H., Sonigara, B., ... & Durak, U. (2023, October). Cybersecurity engineering: Bridging the security gaps in avionics architectures and DO-326A/ED-202A. In *2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)* (pp. 1-8). IEEE.
- [8] Williams, P. A., & Woodward, A. J. (2015). Cybersecurity vulnerabilities in medical devices: a complex environment and multifaceted problem. *Medical Devices: Evidence and Research*, 305-316.
- [9] Cremer, F., Sheehan, B., Fortmann, M., Kia, A. N., Mullins, M., Murphy, F., & Materne, S.



- (2022). Cyber risk and cybersecurity: a systematic review of data availability. The Geneva papers on risk and insurance. Issues and practice, 47(3), 698.
- [10] Rahimi, N., Schuelke-Leech, B.-A., & Mirhassani, M. (2025). Cyber Threat Susceptibility Assessment for Heavy-Duty Vehicles Based on ISO/SAE 21434. IEEE Open Journal of Vehicular Technology. <https://ieeexplore.ieee.org/document/10921673/>
- [11] International Organization for Standardization, & SAE International. (2021). Road vehicles — Cybersecurity engineering (ISO/SAE 21434:2021). Author.
- [12] Ishimatsu, T., Leveson, N., Thomas, J., & Katahira, M. (2010). Modeling and hazard analysis using STPA. <https://dspace.mit.edu/handle/1721.1/79639>
- [13] Schmittner, C., Ma, Z., & Puschner, P. (2016, September). Limitation and improvement of STPA-Sec for safety and security co-analysis. In International Conference on Computer Safety, Reliability, and Security (pp. 195-209). Cham: Springer International Publishing.
- [14] Hossain, Md. I., & Hasan, R. (2023). Improving Security Practices in Health Information Systems with STRIDE Threat Modeling. 2023 IEEE 9th World Forum on Internet of Things (WF-IoT). <https://ieeexplore.ieee.org/document/10539589/>
- [15] Schaad, A., & Garaga, A. (2012). Automating architectural security analysis. <https://dl.acm.org/doi/10.1145/2295136.2295162>
- [16] 윤민희. (2025). STPA-STRIDE 기반 구조적 보안 위협 모델링 프레임워크 제안: 차량 통합제어장치 적용 사례. 한국정보처리학회 학술대회논문집, 32(1), 131-134.
- [17] Mirembe, D., & Muyeba, M. (2008). Threat modeling revisited: Improving expressiveness of attack. <https://ieeexplore.ieee.org/abstract/document/4625253/>
- [18] Daley, K., Larson, R., & Dawkins, J. (2002, December 10). A structural framework for modeling multi-stage network attacks. Proceedings. International Conference on Parallel Processing Workshop. <https://ieeexplore.ieee.org/document/1039705/>
- [19] Ali, A. T., & Gruska, D. P. (2021). Attack Trees with Time Constraints. <https://www.semanticscholar.org/paper/3826a0d35dc0f79e69e44722c36be95ece9dcc9a>



- [20] Kordy, B., Mauw, S., Radomirovic, S., & Schweitzer, P. (2014). Attack-defense trees. *J. Log. Comput.* <https://academic.oup.com/logcom/article-lookup/doi/10.1093/logcom/exs029>
- [21] De Souza, N. P., César, C. D. A. C., de Melo Bezerra, J., & Hirata, C. M. (2020). Extending STPA with STRIDE to identify cybersecurity loss scenarios. *Journal of Information Security and Applications*, 55, 102620.
- [22] Schmittner, C., Ma, Z., & Puschner, P. (2016, September). Limitation and improvement of STPA-Sec for safety and security co-analysis. In *International Conference on Computer Safety, Reliability, and Security* (pp. 195-209). Cham: Springer International Publishing.
- [23] Yu, J., Wagner, S., & Luo, F. (2021). Data-flow-based adaption of the system-theoretic process analysis for security (STPA-sec). *PeerJ Computer Science*, 7, e362.
- [24] Salfer, M., & Eckert, C. (2015). Attack surface and vulnerability assessment of automotive Electronic Control Units. 2015 12th International Joint Conference on E-Business and Telecommunications (ICETE). <https://www.scitepress.org/Link.aspx?doi=10.5220/0005550003170326>
- [25] Pedroza, G., Apvrille, L., & Knorreck, D. (2011). AVATAR: A SysML Environment for the Formal Verification of Safety and Security Properties. 2011 11th Annual International Conference on New Technologies of Distributed Systems. <https://ieeexplore.ieee.org/document/5957992/>
- [27] Yang, Y., Zhang, Y., Liu, W., Li, J., Shi, P., Zhong, D., Yang, J., Chen, T., Cao, S., Ren, Y., Wu, Y., & Zhang, X. (2025). Automating Function-Level TARA for Automotive Full-Lifecycle Security. *ArXiv*. <https://arxiv.org/abs/2504.18083>
- [28] Cho, H., & Kim, S. (2025). Threat Modeling for the Defense Industry: Past, Present, and Future. *IEEE Access*. <https://ieeexplore.ieee.org/abstract/document/10921707/>
- [29] Marksteiner, S., Schmittner, C., Christl, K., Ničković, D., Sjödin, M., & Sirjani, M. (2023). From TARA to Test: Automated Automotive Cybersecurity Test Generation Out of Threat



Modeling. <https://doi.org/10.1145/3631204.3631864>

[30] Cohen, J. (2013). *Statistical power analysis for the behavioral sciences*. routledge.

